

Unit-6

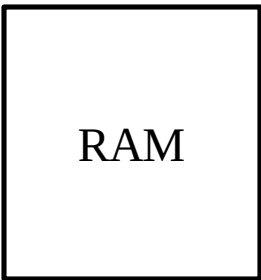
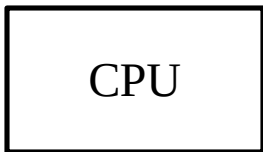
- Data on External Storage
- File Organization and Indexing
- Cluster Indexes, Primary and Secondary Indexes
- Index data Structures
- Hash Based Indexing
- Tree base Indexing
- Comparison of File Organizations
- Indexes and Performance Tuning
- Intuitions for tree Indexes
- Indexed Sequential Access Methods (ISAM)
- B+ Trees: A Dynamic Index Structure.

Data on External Storage

- **Storage:** Offer persistent data storage, data saved on a persistent storage is not lost when the system shutdowns or crashes.
- **Magnetic Disks:** Can retrieve random page at fixed cost.
- **Tapes:** Can only read pages in sequence. Cheaper than disks; used for archival storage.
- Other types of persistent storage devices:
 - Optical storage (CD-R, CD-RW, DVD-R, DVD-RW)
 - Flash Memory.
- Each record in a file has a unique identifier called a record id or rid.

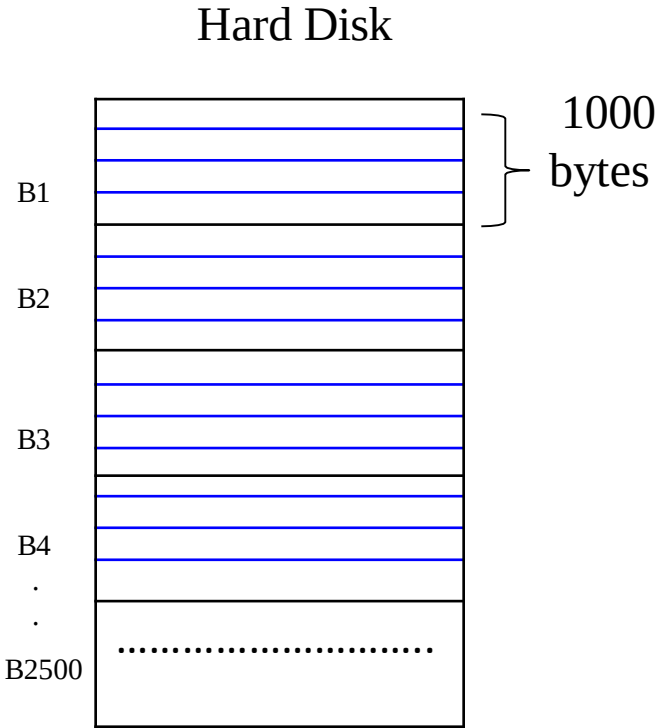
Select * from student where id=1002;

DBMS software



Index Table

Key	Pointer



Example:

Total number of records in student table =10,000

Size of each record= 250 bytes

Size of each block in hard disk= 1000bytes

Total number of records in each block= 1000/250=4 records

Total number of blocks in hard disk= 10,000/4=2,500 blocks

unordered records

Id
201
102
1
6
19
5

Best case = 1

Worst case = 2,500

Average case = 2,500/2
=1250

ordered records

Binary search

$\log_2(N) = 12$

$N = 2,500$

Id
1
2
3
4
5
6

I/O cost increases

- A **record** is a tuple or a row in a table.
 - Fixed-size records or variable-size records
- A **page** is a fixed length block of data for disk I/O.
 - A data page contains a collection of records.
 - A file consists of pages.
- A **file** is a collection of records.
 - Store one table per file, or multiple tables in the same file.
- The unit of information read from or written to disk is page. Typically the size of a page is 4kb or 8kb.

File Organization and Indexing

- Method of arranging a file of records on external storage.
 - *Record id (rid)* is used to locate a record on a disk
 - *Indexes* are data structures to efficiently search rids of given values

Alternative File Organizations and Comparison of File Organizations

- Many alternatives exist, each ideal for some situations, and not so good in others:
 - **1.Heap files:** Records are unsorted. Suitable when typical access is a file scan retrieving all records without any order.
 - Fast update (insertions / deletions)
 - **2.Sorted Files:** Records are sorted. Best if records must be retrieved in some order, or only a 'range' of records is needed.
 - Examples: employees are sorted by age.
 - Slow update in comparison to heap file.

- **3.Indexes:** Data structures to organize records via **trees** or **hashing**.
 - For example, create an index on employee age.
 - Like sorted files, speed up searches for a subset of records that match values in certain (“search key”) fields
 - Updates are much faster than in sorted files.

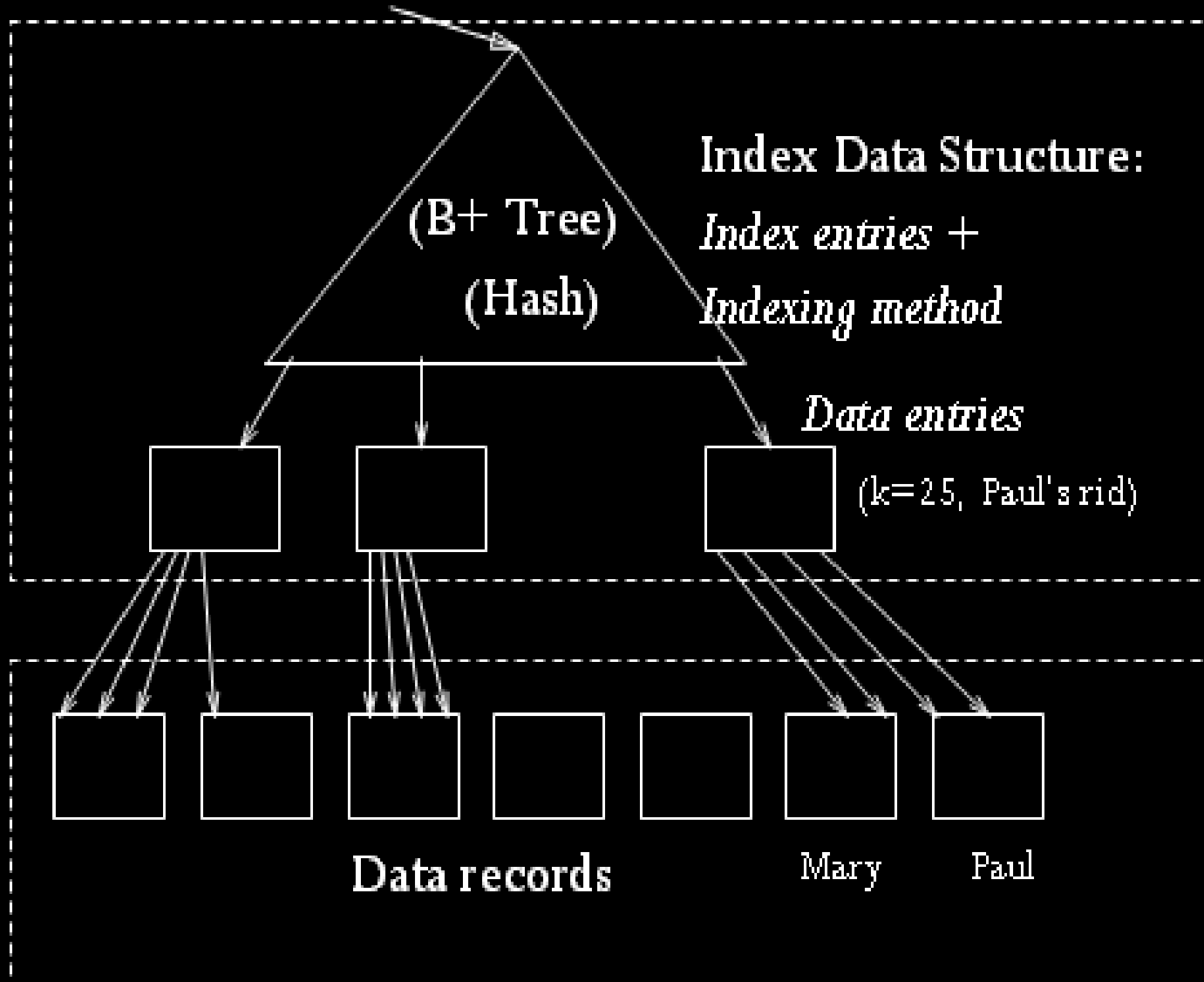
Indexes

- *Indexes* are data structures to efficiently search records of given values
- Any subset of the attributes of a table can be the search key for an index on the relation.
 - Search key does not have to be candidate key
 - Example: employee age is not a candidate key.
- An **index file** contains a collection of data entries (called k^*).

- **Three alternatives** for what to store in a **data entry**:
 - (Alternative 1): Data record with key value **k**
 - Example data record = data entry: <**age**, name, salary>
 - (Alternative 2): <**k**, rid of data record with search key value **k**>
 - Example data entry: <**age**, rid>
 - (Alternative 3): <**k**, list of rids of data records with search key **k**>
 - Example data entry: <**age**, rid_1, rid_2, ...>
- Choice of alternative for data entries is independent of the indexing method.
 - **Indexing method** takes a search key and finds the data entries matching the search key.
 - Examples of indexing methods: **B+ trees** or **hashing**.

Indexing Example

Search key value: find employees with age = 25



Index File
(Small for
efficient
search)

Data File
(Large)

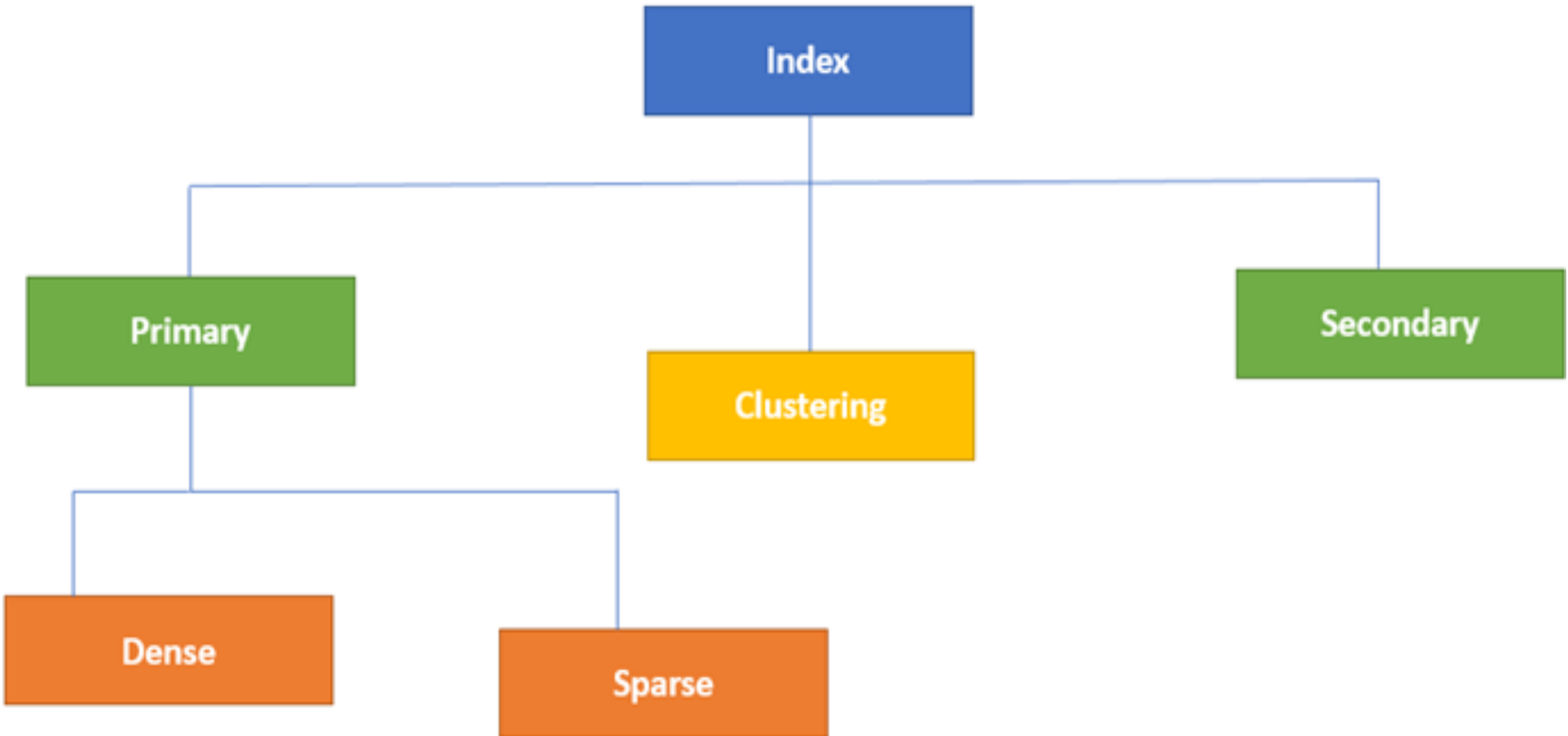
- An Index is a small table having only two columns.
- The first column comprises a copy of the primary or candidate key of a table (or) any subset of the attributes of a table can be the search key for an index on the relation
- Its second column contains a set of pointers for holding the address of the disk block where that specific key value stored.

Key	Pointer

An index:

- Takes a search key as input
- Efficiently returns a collection of matching records.

Types of Index



Primary Index

- If the index is created on the basis of the primary key of the table, then it is known as primary indexing.
- These primary keys are unique to each record and contain 1:1 relation between the records.
- These are stored in sorted order, the performance of the searching operation is quite efficient.
- The primary index can be classified into two types: Dense index and Sparse index.

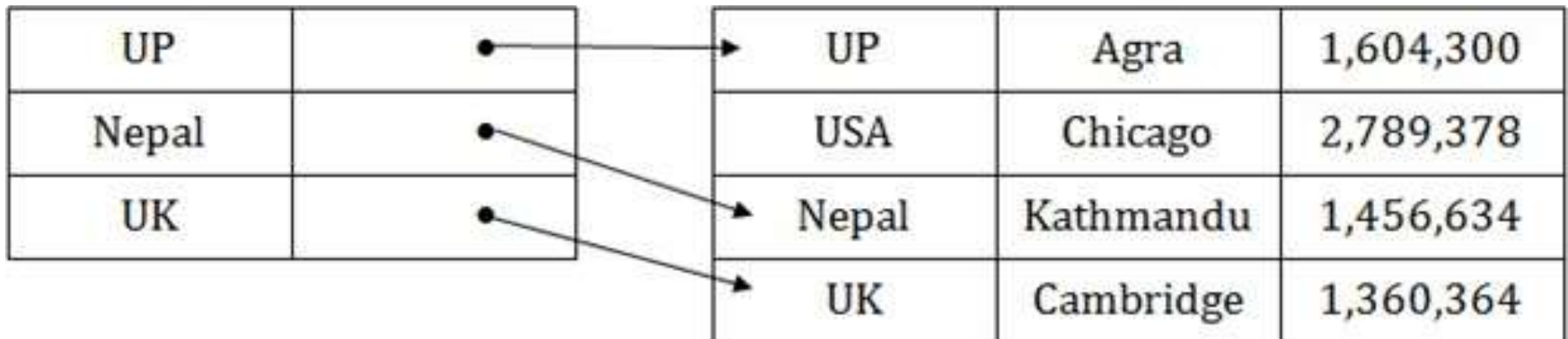
Dense Index

- The dense index contains an index record for **every search key value** in the data file. It makes searching **faster**.
- In this, the number of records in the index table is same as the number of records in the main table.
- It needs **more space** to store index record itself.
- The index records have the search key and a pointer to the actual record on the disk.

UP	•	→	UP	Agra	1,604,300
USA	•	→	USA	Chicago	2,789,378
Nepal	•	→	Nepal	Kathmandu	1,456,634
UK	•	→	UK	Cambridge	1,360,364

Sparse Index

- In the data file, **index record appears only for a few items**. Each item points to a block.
- In this, instead of pointing to each record in the main table, the index points to the records in the main table in a gap.

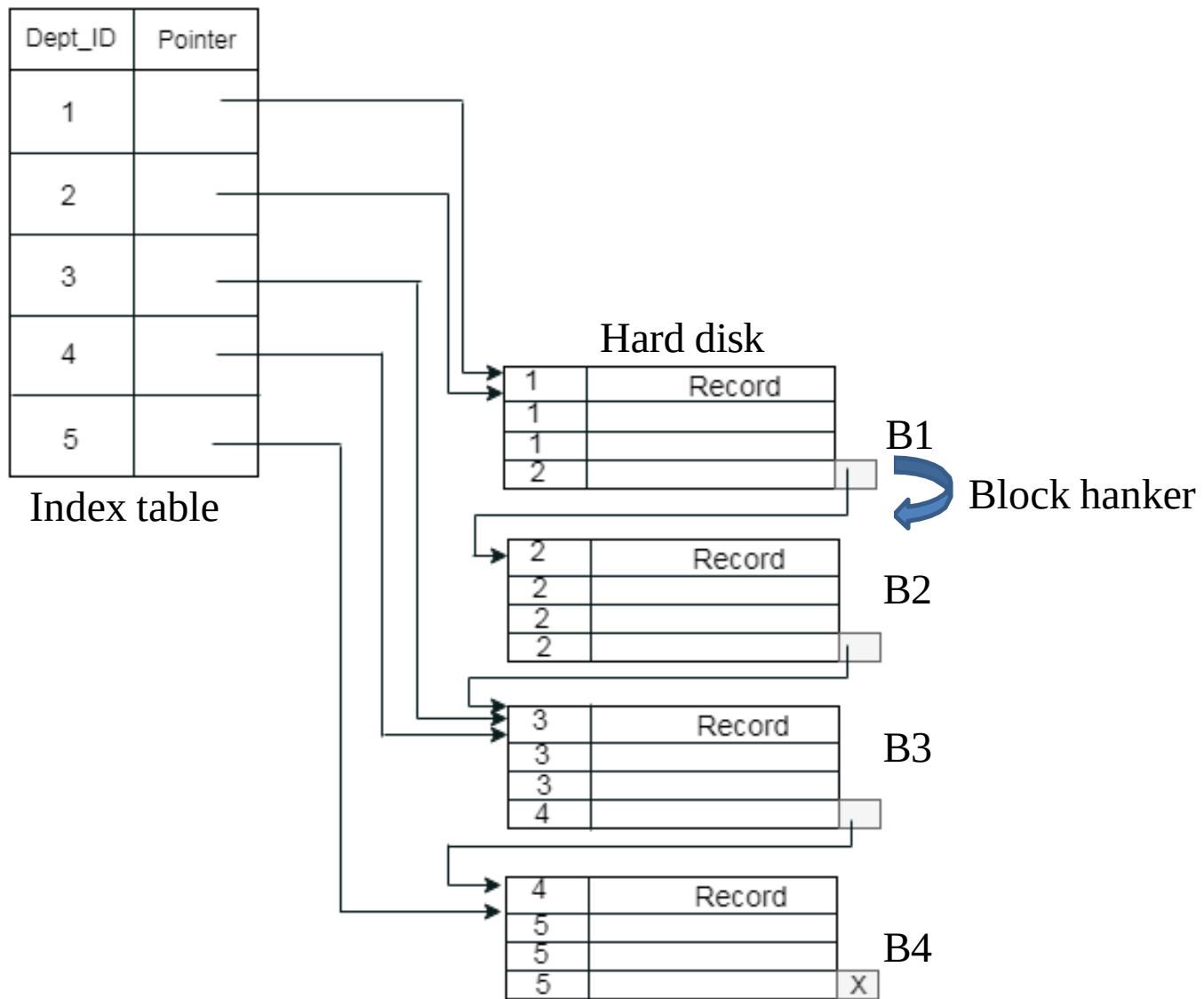


Clustering Index

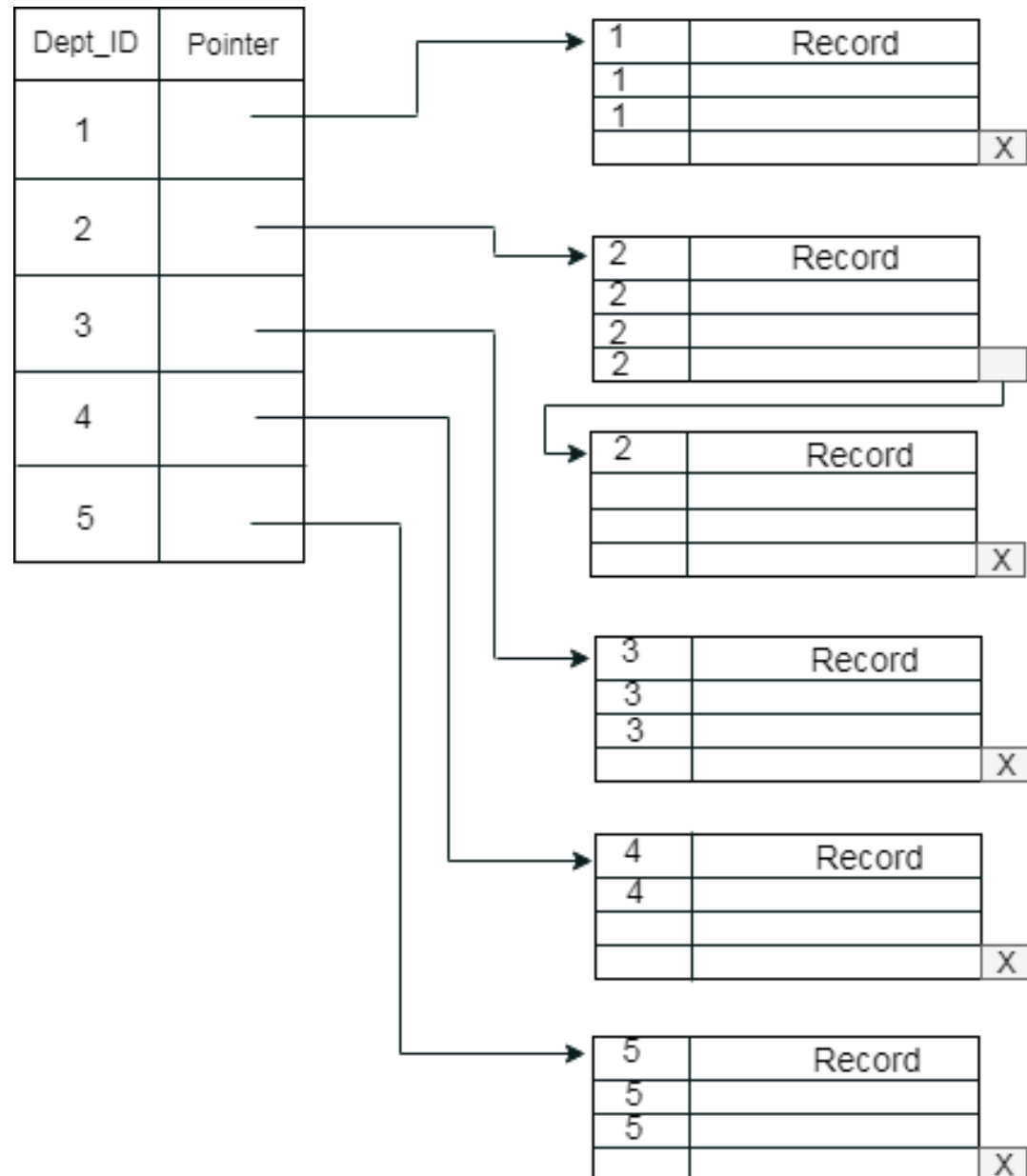
- Clustered index is defined on an ordered data file. The data file is **ordered on a non-key field**.
- In this case, to identify the record faster, we will group two or more columns to get the unique value and create index out of them. This method is called a clustering index.
- The records which have similar characteristics are grouped, and indexes are created for these groups.

Example:

- Suppose a company contains several employees in each department.
- Suppose we use a clustering index, where all employees which belong to the same Dept_ID are considered within a single cluster, and index pointers point to the cluster as a whole.
- Here Dept_Id is a non-unique key.



- The previous schema is little confusing because one disk block is shared by records which belong to the different cluster.
- If we use separate disk block for separate clusters, then it is called better technique.

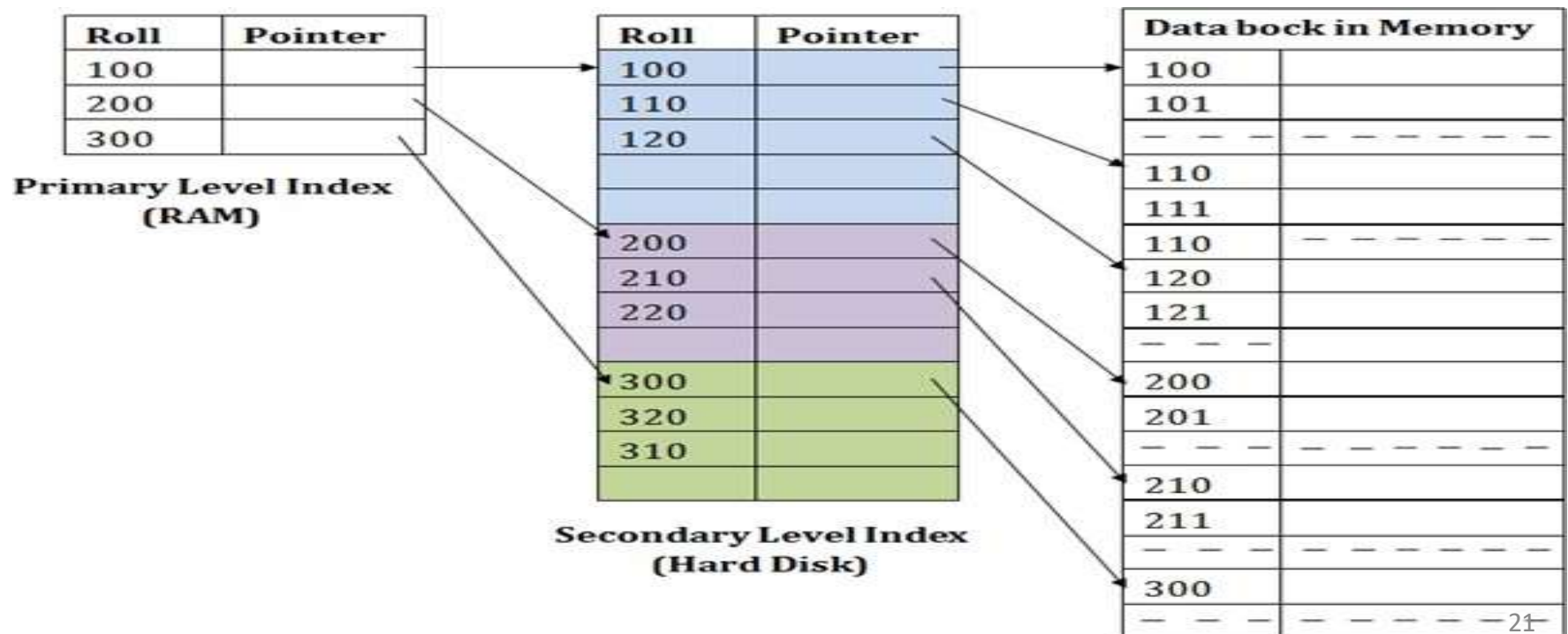


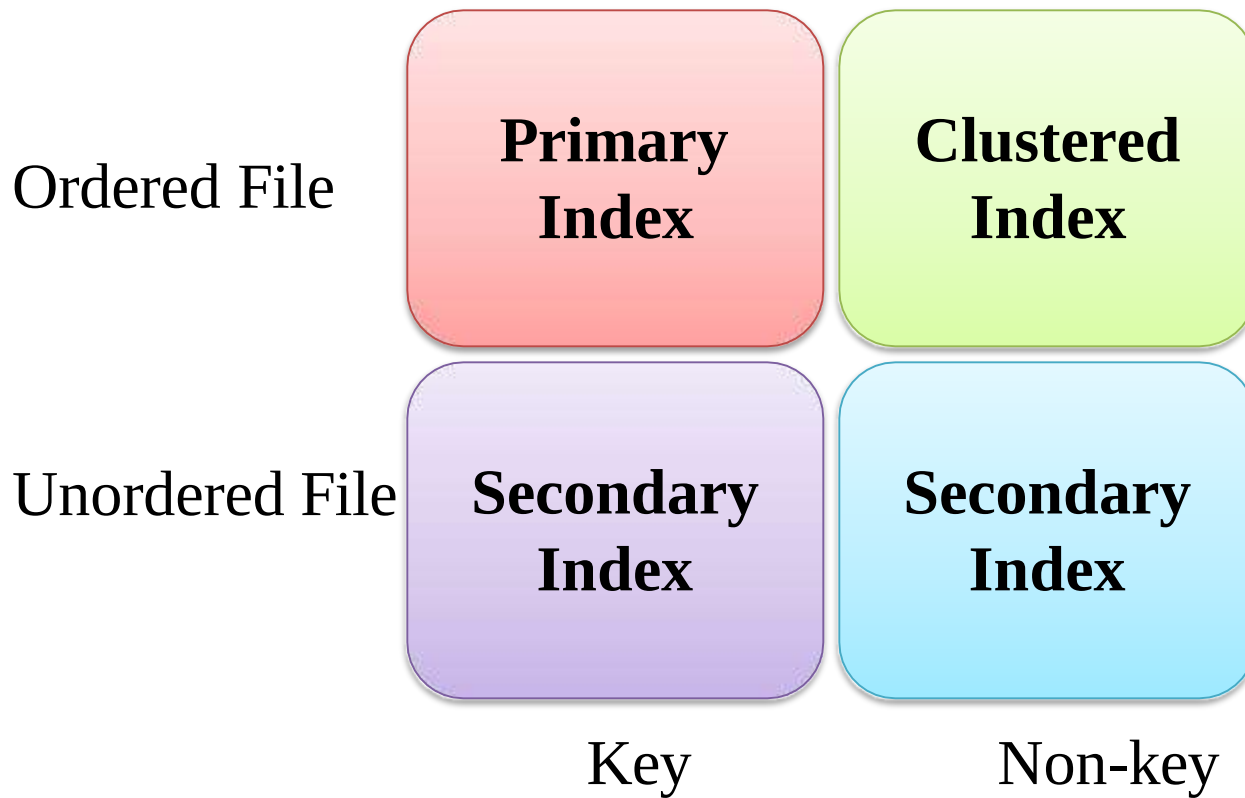
Secondary Index:

- Secondary indexing is a database management technique used to create additional indexes on data stored in a database.
- The main purpose of secondary indexing is to improve the performance of queries and to simplify the search for specific records within a database.
- A secondary index provides an alternate means of accessing data in a database, in addition to the primary index.
- The primary index is typically created when the database is created and is used as the primary means of accessing data in the database.
- Secondary indexes, on the other hand, can be created and dropped at any time, allowing for greater flexibility in managing the database.

- **Secondary Index:** Secondary index may be generated as :

- **Candidate Key Indexing** – When the secondary index is built on a unique field, it allows direct lookups for specific records.
- **Non-Key Indexing** – If built on a field with duplicate values, the secondary index helps in grouping and filtering records efficiently.
- **Multi-Level Indexing** – To further reduce mapping size, an additional indexing layer is introduced, making queries faster and reducing storage overhead.





Index data Structures

- One way to organize data entries is to hash data entries on the search key.
- Another way to organize data entries is to build a tree-like data structure that directs a search for data entries.
- Hash Based Indexing
- Tree based Indexing

Hash-Based Indexing

In hash-based indexing, a hash function is used to convert a key into a hash code. This hash code serves as an index where the value associated with that key is stored. The goal is to distribute the keys uniformly across an array, so that access time is, on average, constant.

Buckets

In hash-based indexing, the data space is divided into a fixed number of slots known as "buckets." A bucket usually contains a single page (also known as a block), but it may have additional pages linked in a chain if the primary page becomes full. This is known as overflow.

Hash Function

The hash function is a mapping function that takes the search key as an input and returns the bucket number where the record should be located. Hash functions aim to distribute records uniformly across buckets to minimize the number of collisions (two different keys hashing to the same bucket).

Disk I/O Efficiency

Hash-based indexing is particularly efficient when it comes to disk I/O operations. Given a search key, the hash function quickly identifies the bucket (and thereby the disk page) where the desired record is located. This often requires only one or two disk I/Os, making the retrieval process very fast.

Insert Operations

When a new record is inserted into the dataset, its search key is hashed to find the appropriate bucket. If the primary page of the bucket is full, an additional overflow page is allocated and linked to the primary page. The new record is then stored on this overflow page.

Search Operations

To find a record with a specific search key, the hash function is applied to the search key to identify the bucket. All pages (primary and overflow) in that bucket are then examined to find the desired record.

Limitations

Hash-based indexing is not suitable for range queries or when the search key is not known. In such cases, a full scan of all pages is required, which is resource-intensive.

Pros of Hash-Based Indexing

- Extremely fast for exact match queries.
- Well-suited for equality comparisons.

Cons of Hash-Based Indexing

- Not suitable for range queries (e.g., "SELECT * FROM table WHERE age BETWEEN 20 AND 30").
- Performance can be severely affected by poor hash functions or a large number of collisions.

Hash-Based Indexing Example

Let's consider a simple example using employee names as the search key.

Employee Records

Name	Age	Salary
Alice	28	50000
Bob	35	60000
Carol	40	70000

Hash Function: $H(x) = \text{ASCII value of first letter of the name} \bmod 3$

- **Alice:** $65 \bmod 3 = 2$
- **Bob:** $66 \bmod 3 = 0$
- **Carol:** $67 \bmod 3 = 1$

Buckets:

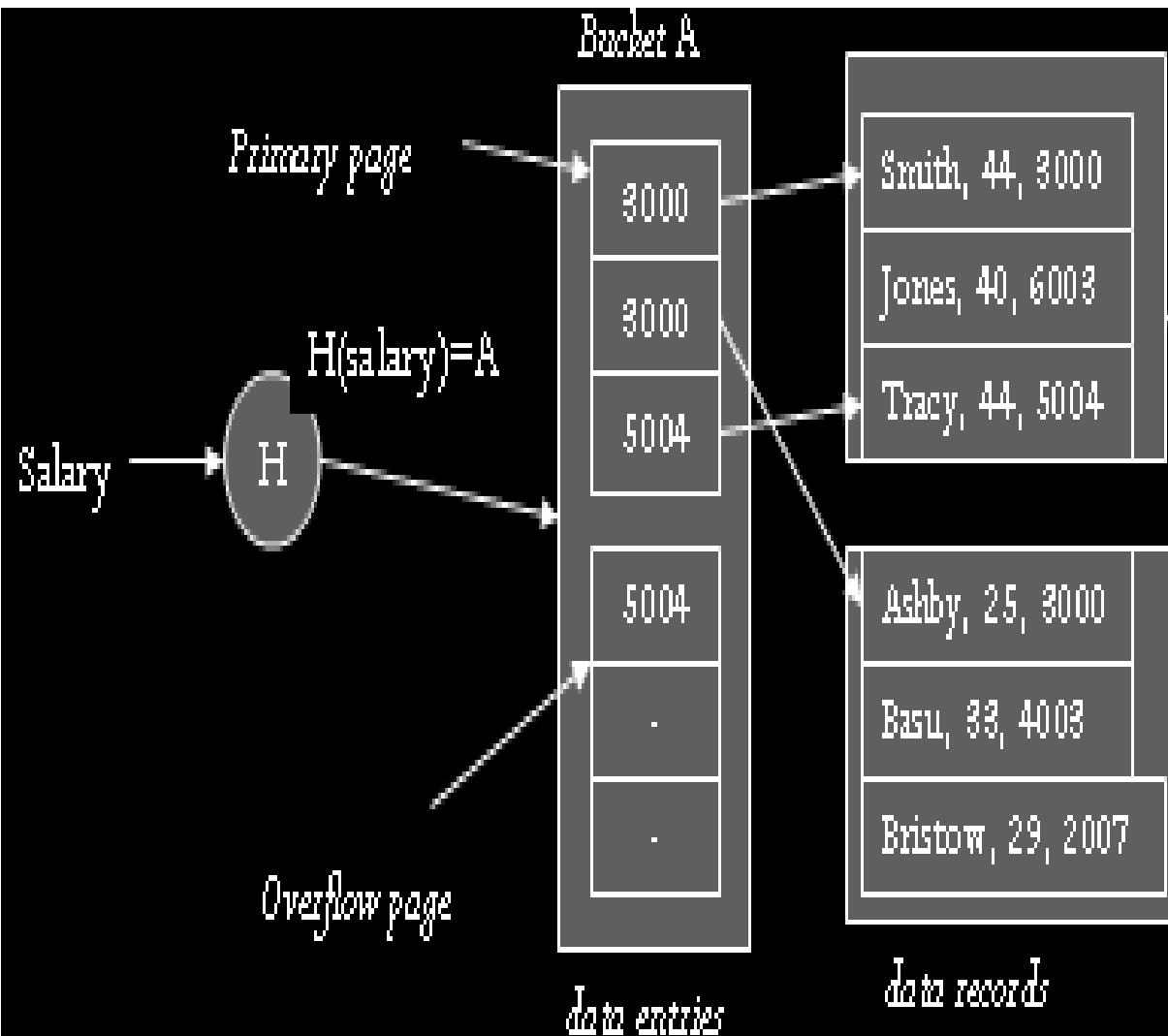
Bucket 0: Bob

Bucket 1: Carol

Bucket 2: Alice

Hash-Based Indexing

- Good for equality selections.
 - Data entries (key, rid) are grouped into buckets.
 - Bucket = *primary* page plus zero or more *overflow* pages.
 - *Hashing function* **h**: $\mathbf{h}(r)$ = bucket in which record r belongs. **h** looks at the *search key* fields of r .



- Search on key value:
 - Apply key value to the hash function -> bucket number
 - Retrieve the primary page of the bucket. Search records in the primary page. If not found, search the overflow pages.
 - Cost of locating rids: # pages in bucket (small)
- Insert a record:
 - Apply key value to the hash function -> bucket number
 - If all (primary & overflow) pages in that bucket are full, allocate a new overflow page.
 - Cost: similar to search.
- Delete a record
 - Cost: Similar to search

Tree-based Indexing

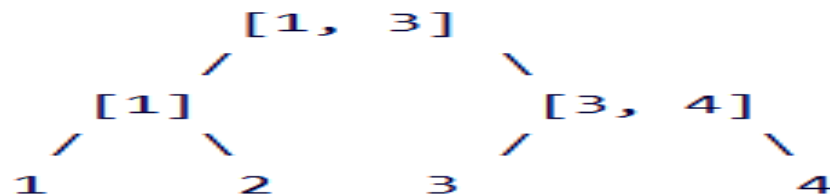
The most commonly used tree-based index structure is the B-Tree, and its variations like B+ Trees and B* Trees. In tree-based indexing, data is organized into a tree-like structure. Each node represents a range of key values, and leaf nodes contain the actual data or pointers to the data.

Tree-based Indexing Example

Continuing with the "Students" table:

ID	Name
1	Abhi
2	Bharath
3	Chinni
4	Devid

A simplified B-Tree index could look like this:



Pros of Tree-based Indexing:

- Efficient for range queries.
- Good for both exact and partial matches.
- Keeps data sorted.

Cons of Tree-based Indexing:

- Slower than hash-based indexing for exact queries.
- More complex to implement and maintain.

Comparison of File Organizations, Indexes and Performance Tuning

File Organizations

- 1. Objective:** To physically store records on storage media in an organized manner.
- 2. Methodologies:** Includes sequential, random (or direct), and hashed file organizations, among others.
- 3. Implications:**
 - Sequential organization is suitable for batch processing but inefficient for random access.
 - Direct or random organization allows fast access but can be inefficient in terms of storage space.
 - Hashed file organization is excellent for equality searches but not for range-based queries.
- 4. Real-world Examples:** Ledger systems, log files, archival systems.

Indexing

1. Objective: To create a data structure that improves the speed of data retrieval operations.

2. Methodologies: Includes clustered, non-clustered, primary, secondary, composite, bitmap, and hash indexes, among others.

3. Implications:

- Clustered indexes are excellent for range-based queries but slow down insert/update operations.
- Non-clustered indexes improve data retrieval speed but can take up additional storage.
- Bitmap indexes are useful for low-cardinality columns.

4. Real-world Examples: Search engines, e-commerce websites, any application that requires fast data retrieval.

Performance Tuning

1. Objective: To optimize the resources used by the database for efficient transaction processing.

2. Methodologies: Query optimization, index tuning, denormalization, database sharding, caching, partitioning, etc.

3. Implications:

- Query optimization can dramatically reduce the resources needed for query processing.
- Proper indexing can mitigate the need for full-table scans.
- Denormalization and caching can improve read operations but may compromise data integrity or consistency.

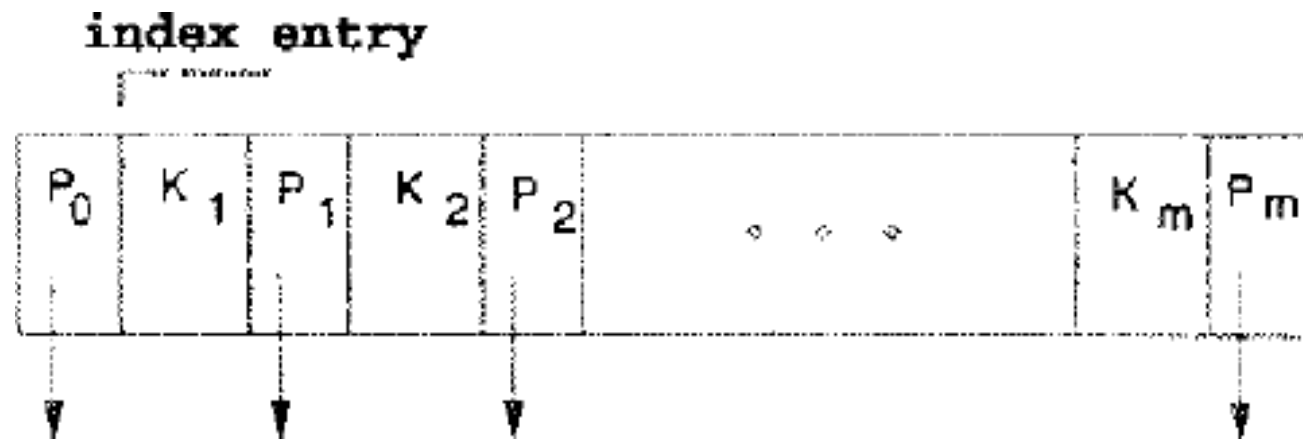
4. Real-world Examples: Financial trading systems, real-time analytics, high-performance computing.

Indexes and Performance Tuning

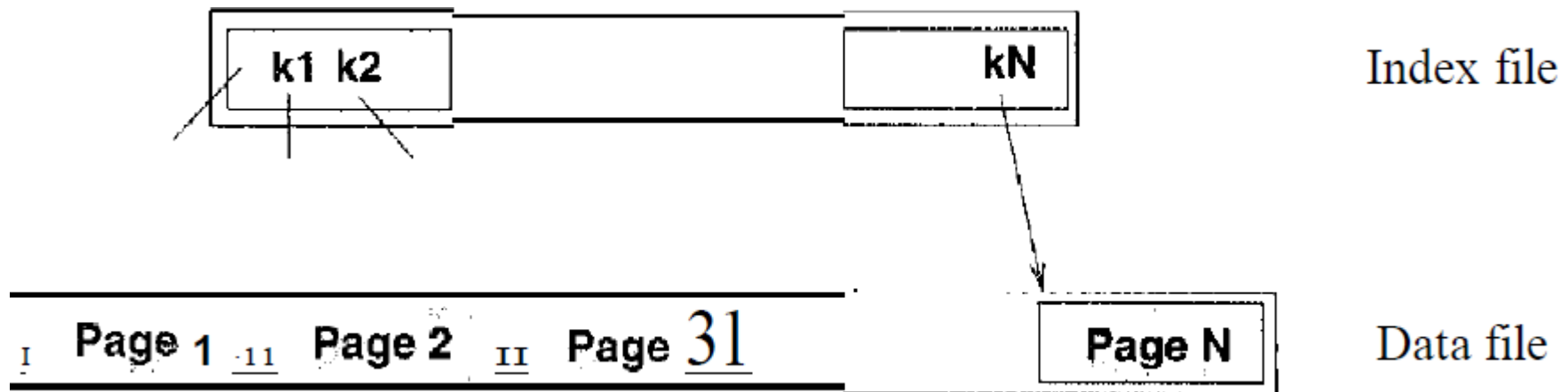
- **Database tuning** describes a group of activities used to optimize and homogenize the **performance** of a **database**
- **Database tuning** aims to maximize use of system resources to perform work as efficiently and rapidly as possible.

Intuitions for tree Indexes

- Find all students with a gpa higher than 3.0
- Binary search
- If the file is large, the initial binary search can be quite expensive, since cost is proportional to the number of pages fetched.
- (*first key on page, pointer to page*), We refer to pairs of the form (*key, pointer*) as index entries.



Format of an index page



One level Index structure

- The potential large size of the index file motivates the tree indexing idea.
- This repeated construction of a one-level index leads to a tree structure with several levels of non-leaf pages.

Tree based Indexing

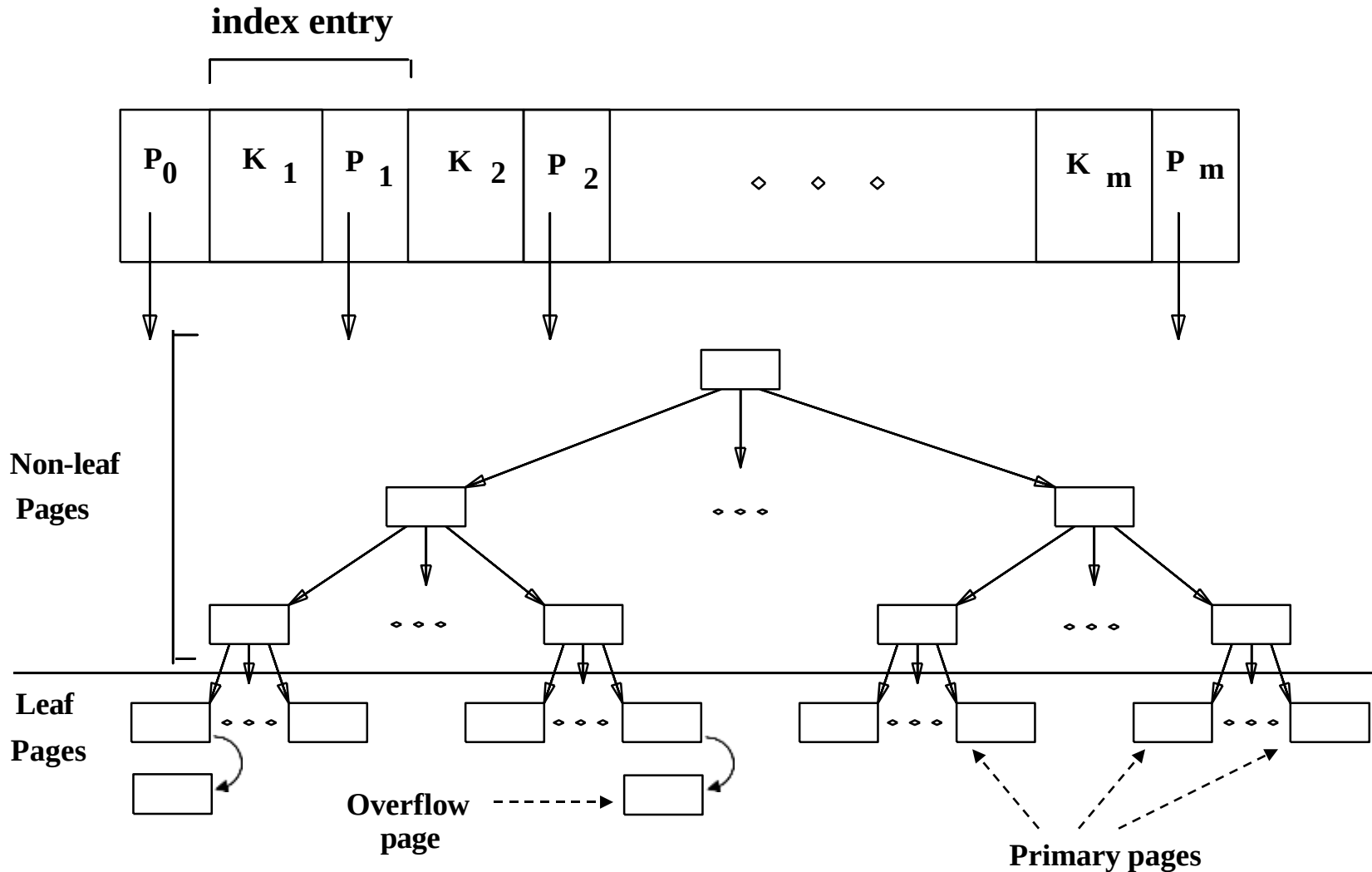
- Tree-structured indexing techniques support both range searches and equality searches
- Indexed Sequential Access Method (ISAM):
static structure;
- *B+ tree*: dynamic, adjusts gracefully under inserts and deletes.

INDEXED SEQUENTIAL ACCESS METHOD (ISAM)

- What is ISAM?
- Structure of ISAM Tree
- Format of Index nodes in ISAM Tree
- How to perform the basic operations in ISAM tree?

Indexed Sequential Access Method (ISAM) is a special file organization solution that is implemented in database systems for the purpose of data retrieval acceleration and process optimization.

Indexed Sequential Access Method



*Non-leaf pages contain **index entries**. Leaf pages contain **data entries**.*

- *Index entries*: <search key value, page id>;
 `direct' search for *data entries*, which are in leaf pages.
- Search: Start at root; use key comparisons to go to leaf.
- Insert: Find leaf that data entry belongs to, and put it there, which may be in the primary or overflow area.
- Delete: Find and remove from leaf; if overflow page is empty, de-allocate.

Static tree structure: *inserts/deletes affect only leaf pages.*

- Frequent updates may cause the structure to degrade
 - Index pages never change
 - some range of values may have too many overflow pages
- e.g., inserting many values between 40 and 51.

- **Creation of ISAM structure**
- **Search Operation in ISAM**
- **Insertion operation in ISAM**
- **Deletion operation in ISAM**
- **Advantages & Disadvantages of ISAM**

It follows bottom up approach.

Steps to be followed for creating ISAM tree structure

1. Sort the data file on the search key field
2. Create the index pages for leaf level.
3. If there are more than one index pages , repeat until the top most index level (root) contains only one page.

Given records

15* 10* 27* 20* 37* 33* 46* 40* 55* 52* 97* 63*

Sorted records

10* 15* 20* 27* 33* 37* 40* 46* 52* 55* 63* 97*

Let us assume node size of ISAM tree is 2.

■

Step 1

Data Pages

10* 15* 20* 27* 33* 37* 40* 46* 52* 55* 63* 97*

STEP 2:

Step 2

Index Pages

20 33

52 63

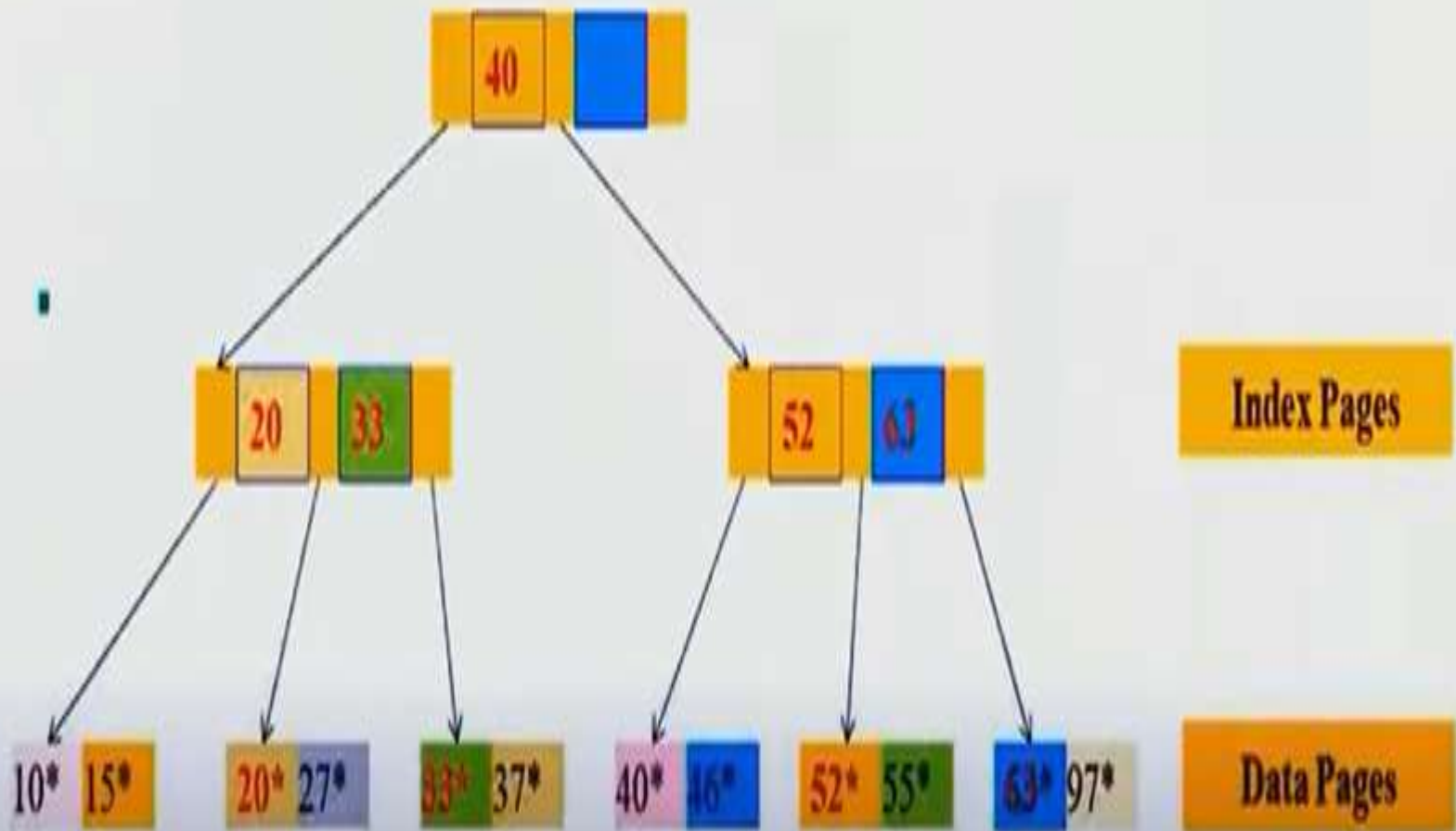
Step 1

Data Pages

10* 15* 20* 27* 33* 37* 40* 46* 52* 55* 63* 97*

node size = 2

ISAM Tree



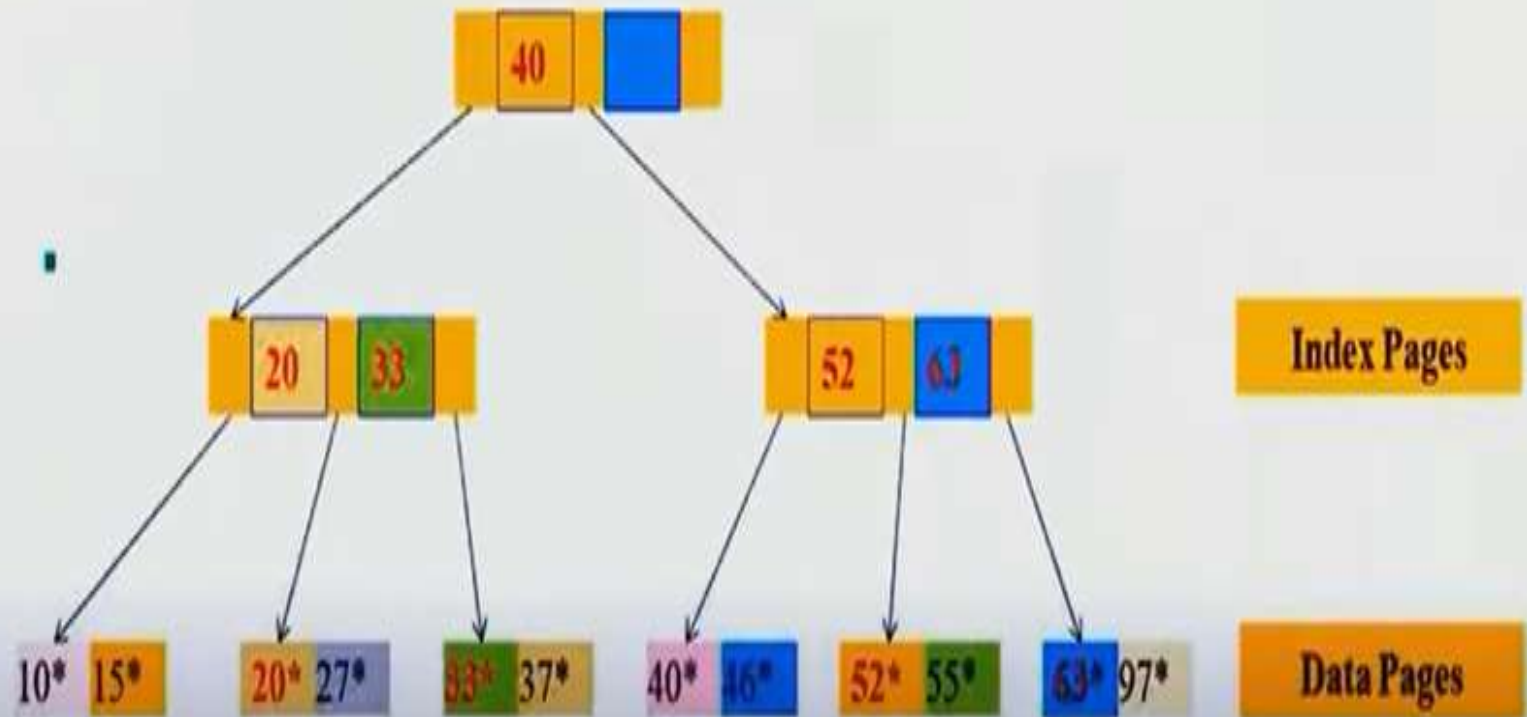
Search operation in ISAM

1. Start at the root index node.
2. Use key comparisons with the search keys in the root index node.
3. If the search key is less than the index key, follow the left pointer pointing to the left child; if it is greater than or equal to the index key, follow the right pointer pointing to the right child.
4. Continue this process at each subsequent index node until you reach a leaf node.
5. Search with in the corresponding data page for the target data.

Search 20

node size = 2

ISAM Tree



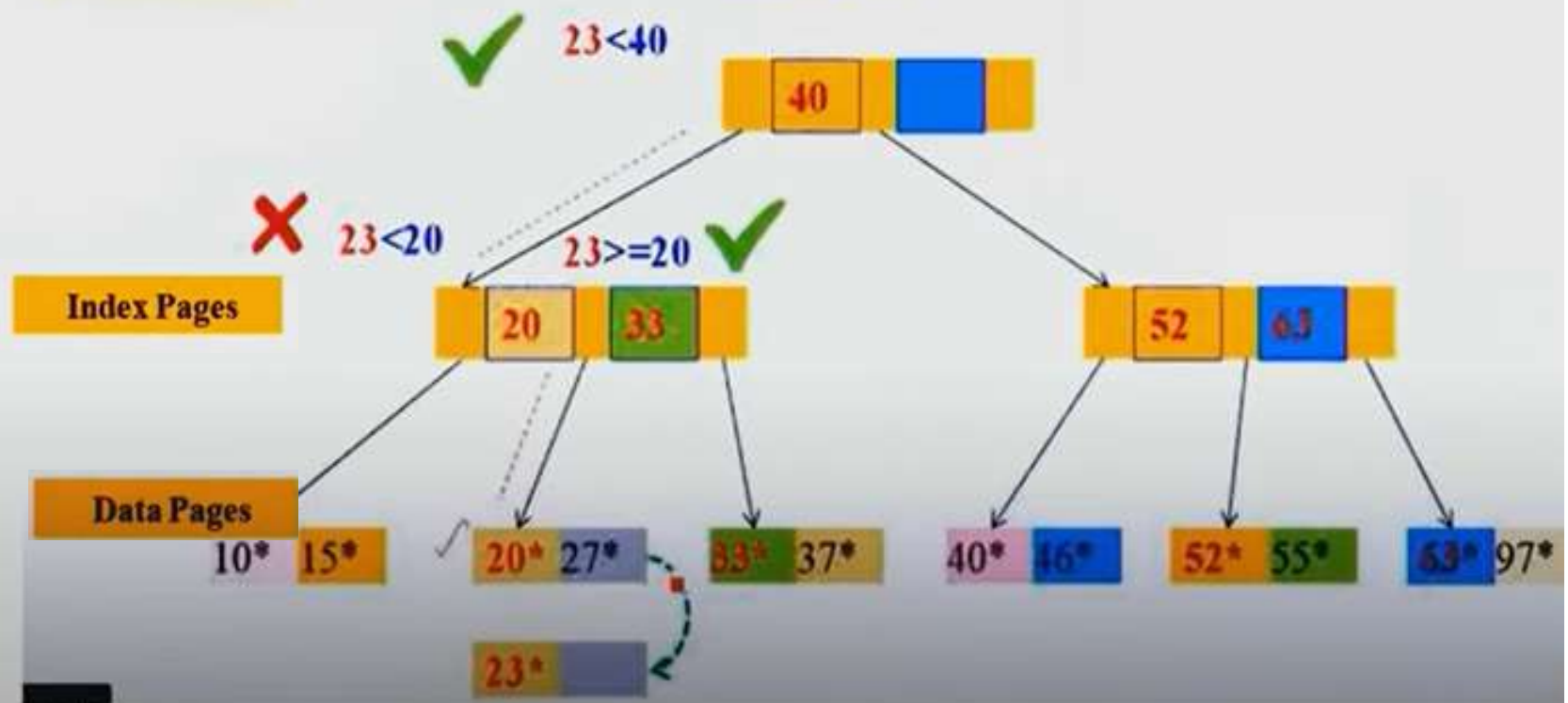
Insertion operation in ISAM

1. If there is space in the corresponding primary data page, insert record in that page
2. Else, Overflow data page needs to be added

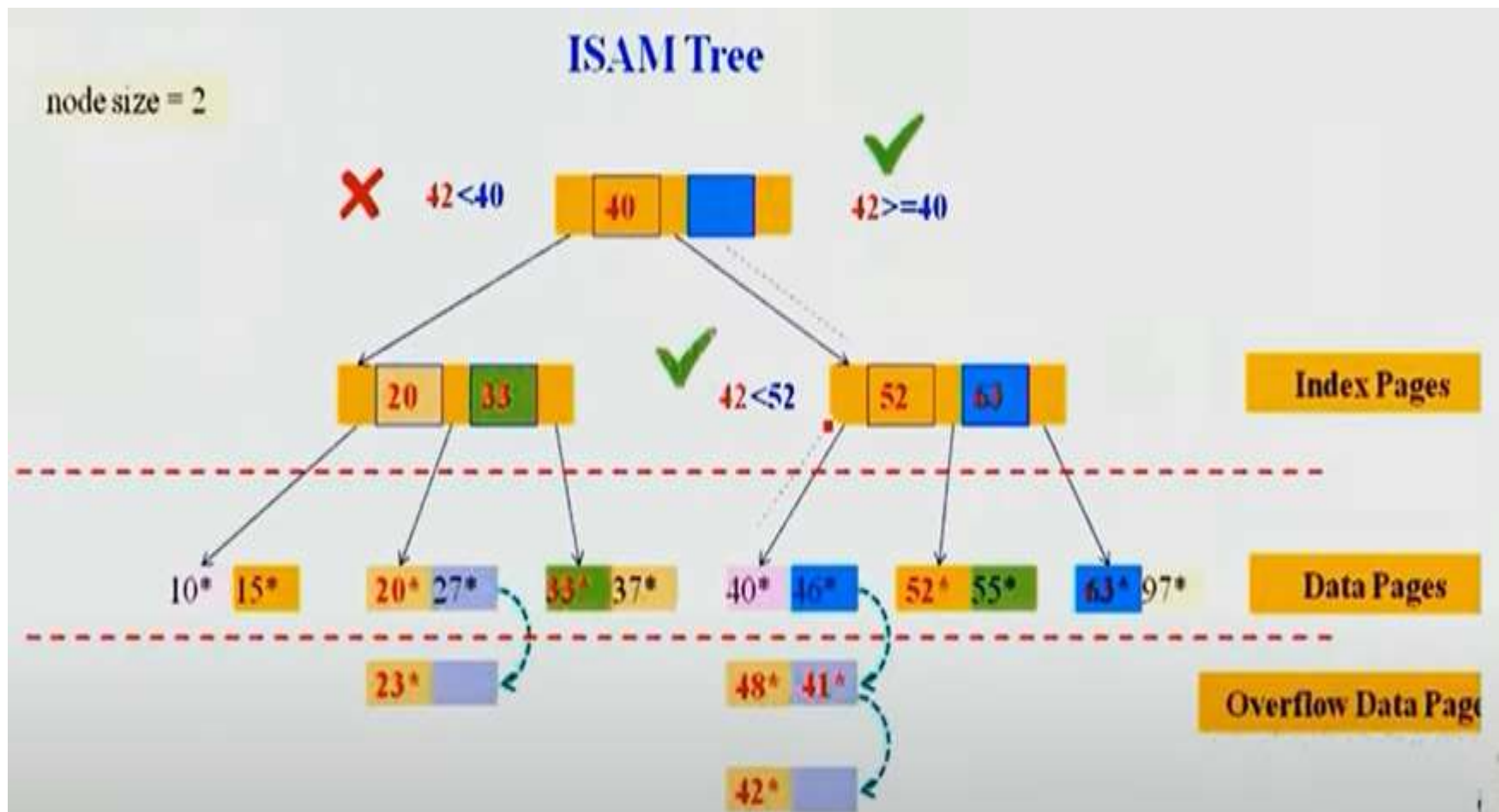
Insert 23

node size = 2

ISAM Tree



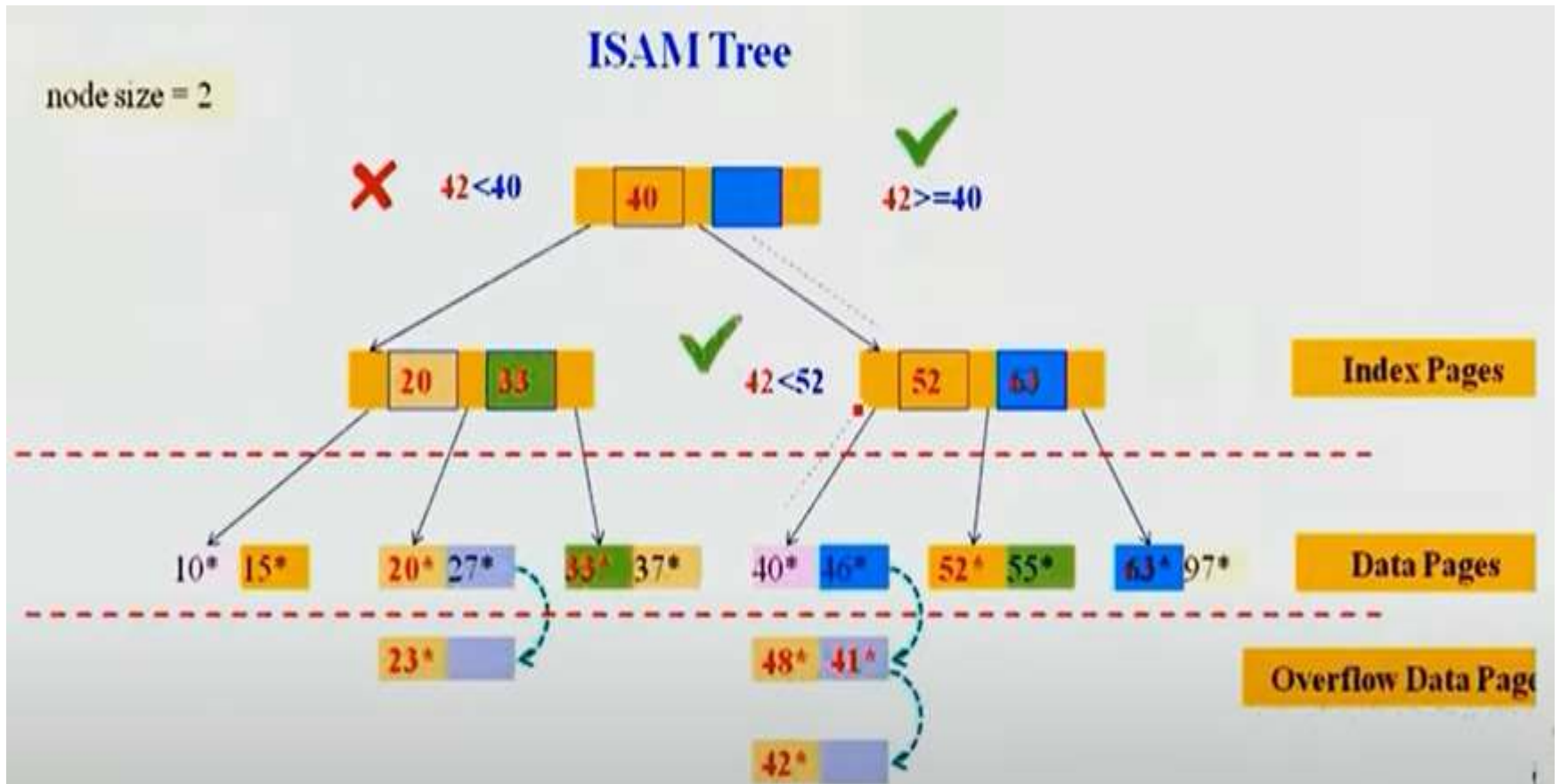
Insert 48,41,42



Deletion operation in ISAM

1. Simply remove the record from the corresponding data page.
2. If the removal makes an overflow data page empty, remove that overflow data page.
3. If the removal makes a primary data page empty, keep it as a placeholder for future insertions.
4. Don't move records from overflow data pages to primary data pages.

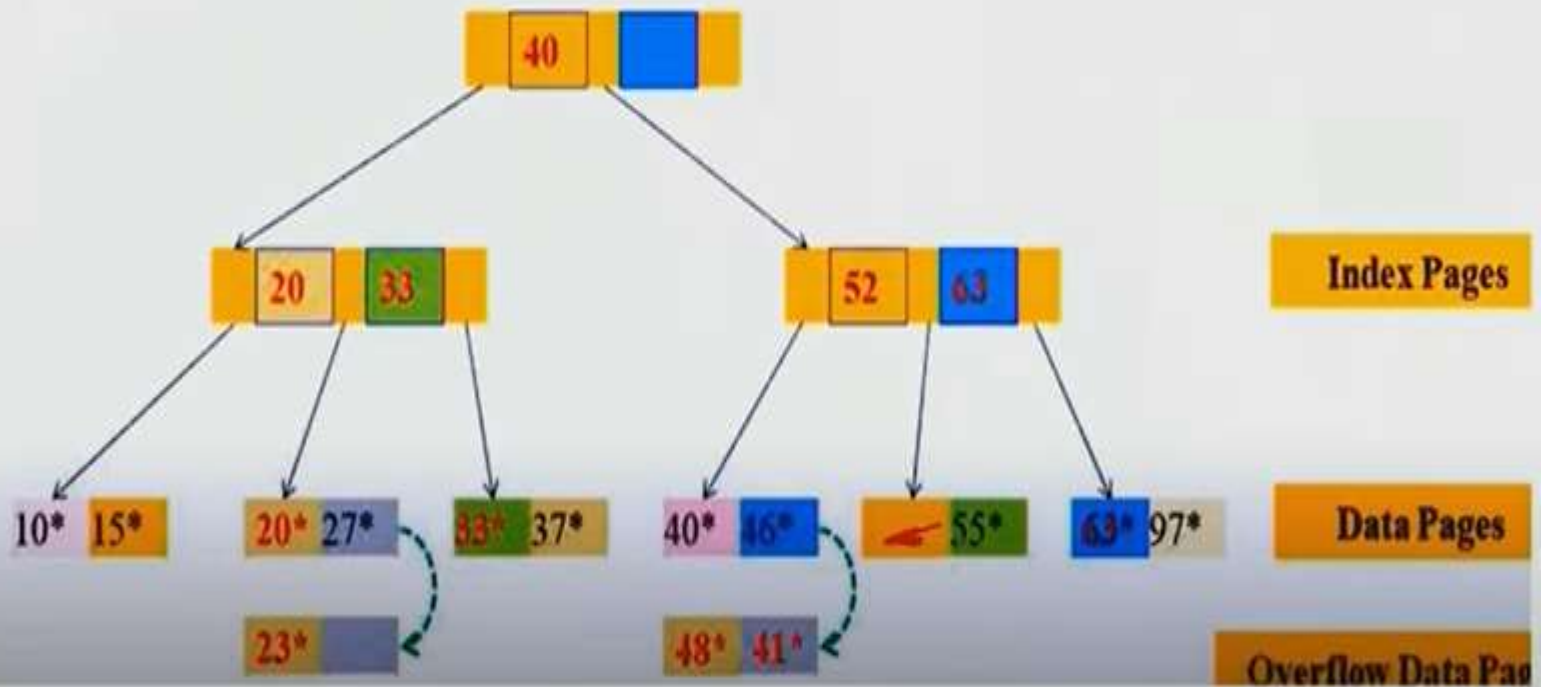
Delete 42



After Deleting 52

node size = 2

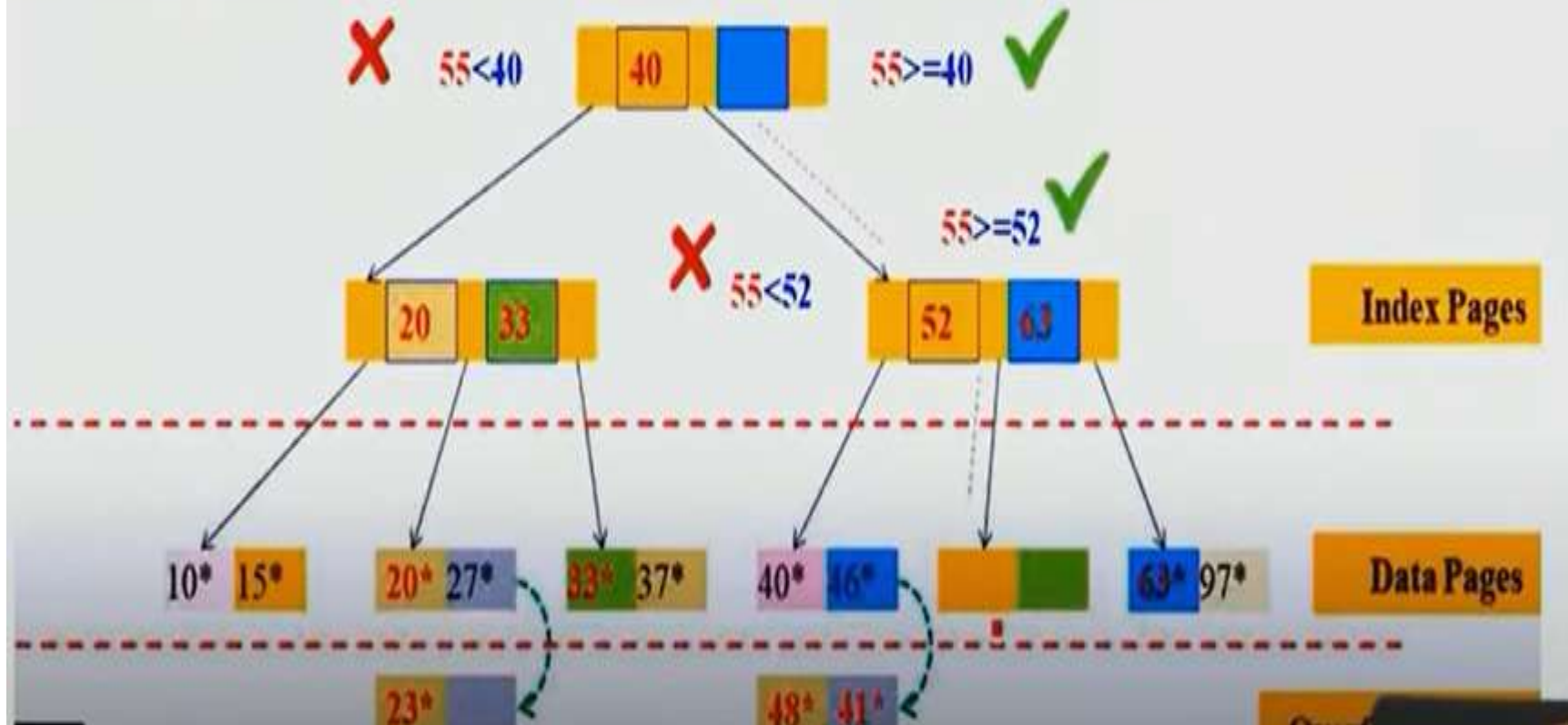
ISAM Tree



After deleting 55

node size = 2

ISAM Tree



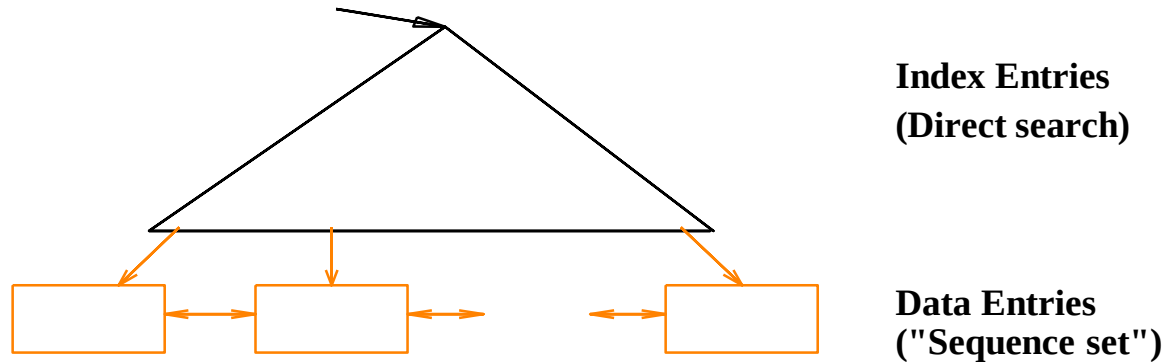
Advantages & Disadvantages of ISAM

Advantage: It performs well when data doesn't change frequently and there are no large overflow chains.

Disadvantage: When more records are inserted, non-sequential overflow chains may be created, which degrades search performance

B+ Tree

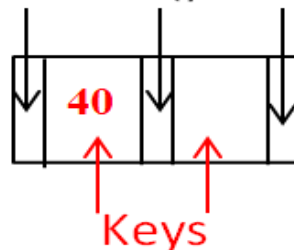
- Dynamic structure - can be updated without using overflow pages!
- Main characteristics:
 - Minimum 50% occupancy (except for root).
 - Supports equality and range-searches efficiently.



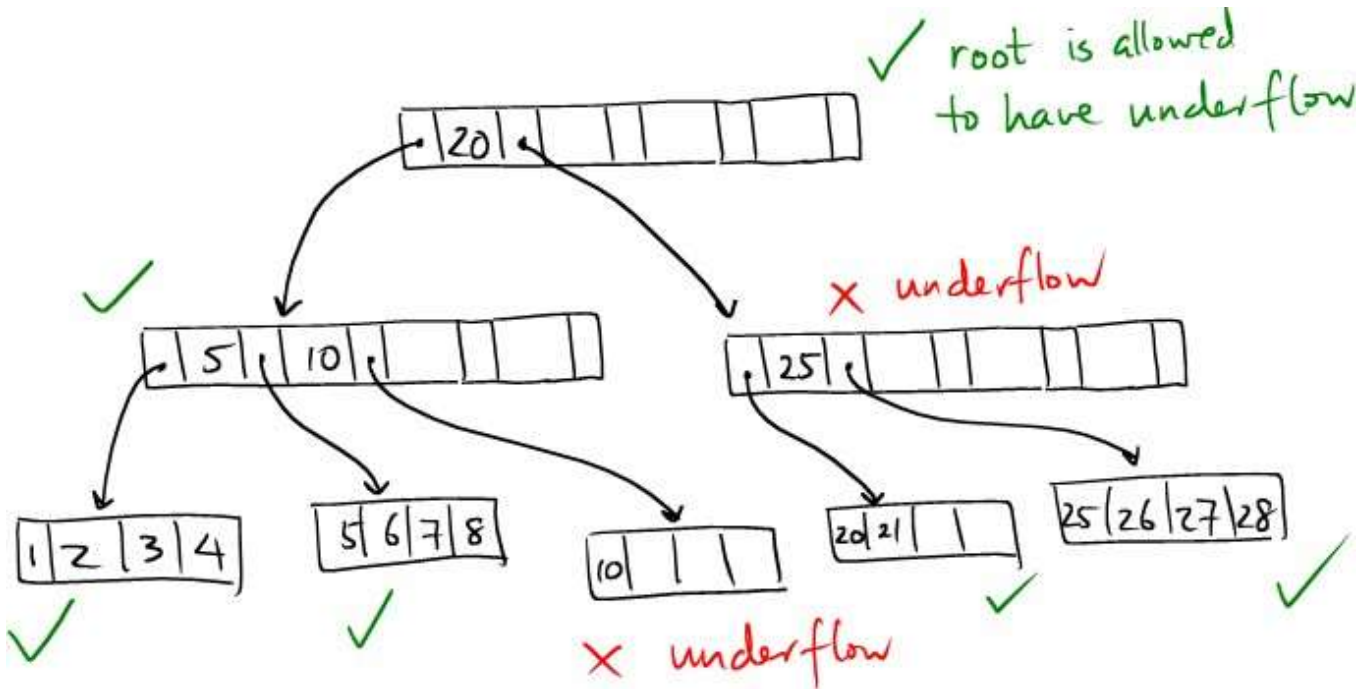
B+ tree with order m

- Root node should have two children (at least 1 search key)
- Other nodes should have minimum $\text{ceil}(m/2)$ children (at least $\text{ceil}(m/2) - 1$ search keys)

Children (pointers)



Example:



Inserting a Data Entry into a B+ Tree

- Find correct leaf L .
- Put data entry onto L .
 - If L has enough space, *done!*
 - Else, must split L (into L and a new node $L2$)
 - Redistribute entries evenly, copy up middle key.
 - Insert index entry pointing to $L2$ into parent of L .
- This can happen recursively
 - To split index node, redistribute entries evenly, but push up middle key. (Contrast with leaf splits.)
- Splits “grow” tree; root split increases height.
 - Tree growth: gets wider or one level taller at top.

The tree distinct cases are:

1. the target node has available space for one more key
2. the target node is full, but its parent has space for one more key
3. the target node and its parent are both full.

B+ Tree

- ❖ The B+ tree file organization is a method to organize a file in **DBMS**. The records are sorted using the primary key to conform with the key-value pair rule. It is in a tree-like structure.
- ❖ B+ tree file organization is much similar to the **BST** (Binary Search Tree), but it can have more than two children. The leaf nodes of the B+ tree file organization contain all the records, and the middle nodes contain only pointers to the leaf node. It doesn't carry any records

1. Data is present in leaf nodes only.
2. Tree can be left biased/right biased.
3. Left subtree must be less than root node and right subtree will be greater than root node.
4. All leaf nodes are connected with linked lists.

Note: If order of tree is m , then

Max children= m

Min children= $\lceil m/2 \rceil$

Max keys= $m-1$

Min keys= $\lceil m/2 \rceil - 1$

Insert 1,4,7,10,17,21,31,25,19,20,28,42 of order 4.

Sol: Order= $m=4$

Max children= 4

Min children= $\lceil m/2 \rceil = 4/2 = 2$

Max keys= $m-1 = 4-1 = 3$

Min keys= $\lceil m/2 \rceil - 1 = 1$

Insert 1,4,7.

B+ Tree Example:-

Insert 1, 4, 7, 10, 17, 21, 31, 25, 19, 20, 28, 42

Sol:- Order = $m = 4$

Max. children = 4

Min. children = $\lceil 4/2 \rceil = 2$

Max. keys = $m - 1 = 4 - 1 = 3$

Min. keys = $\lceil m/2 \rceil - 1 = 1$

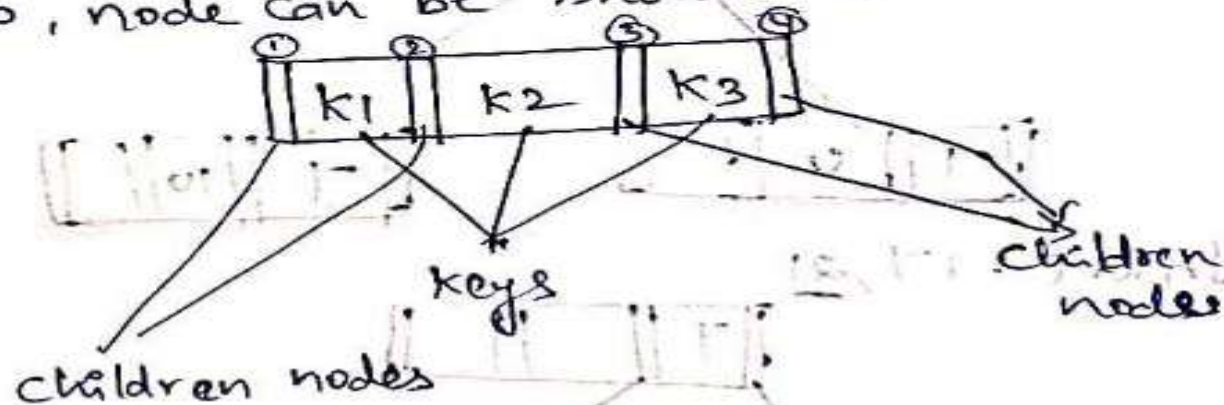
Note:-

Ceil function
 $\lceil 2.3 \rceil = 3$

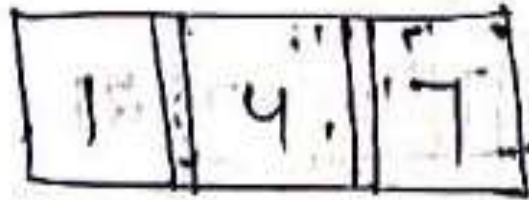
Insert:-

As keys can be inserted maximum only 3.

So, node can be shown as



Insert: 1, 4, 7



Insert: 10 $\Rightarrow$

1	4	7
---	---	---

 10

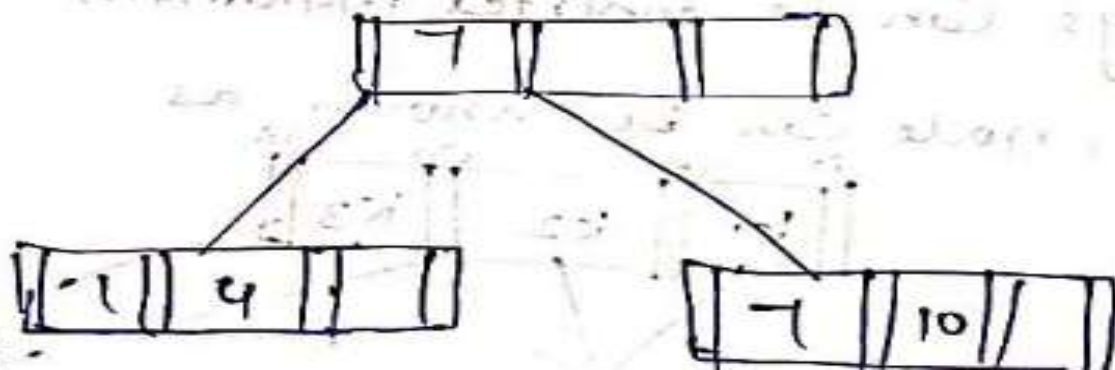
As node is full, split the node, middle elements (either left element or right element) moves to upper node.

→ If left element moves, then it is left biased and in entire solution we have to follow left bias only.

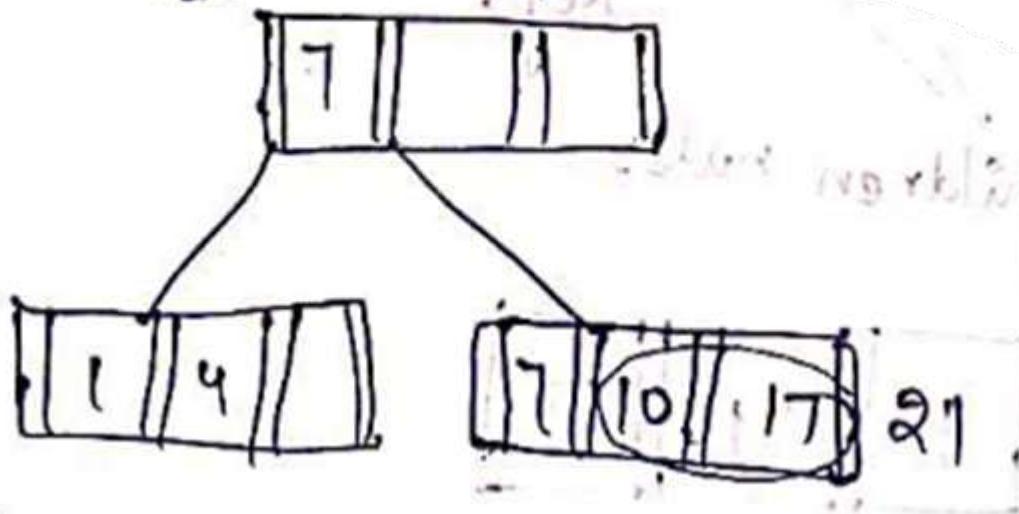
→ If Right element moves, then it is right biased and in entire solution we have to follow right bias only.

Insert: 10

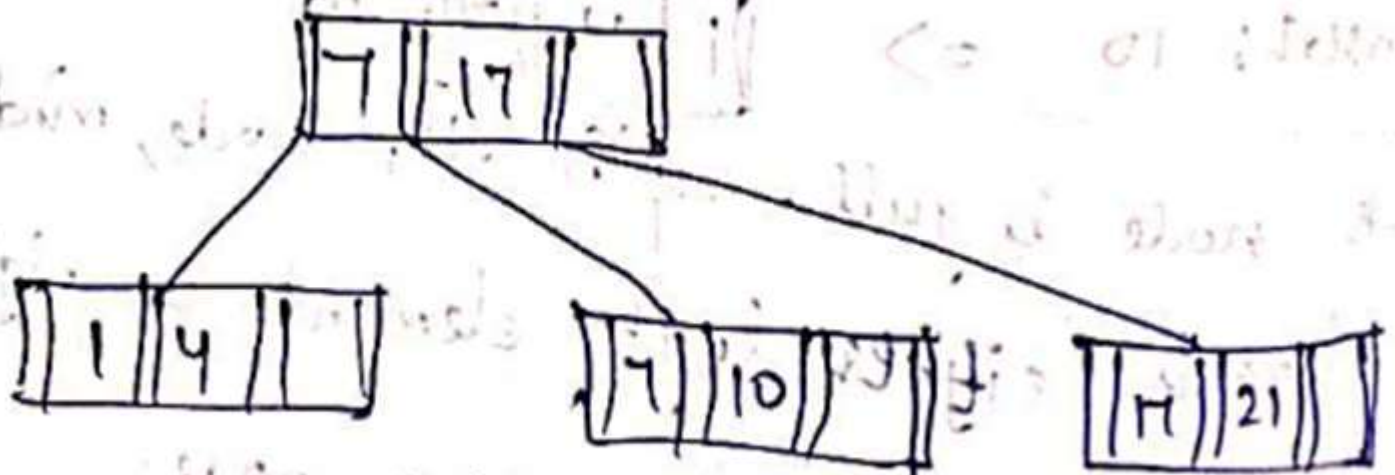
Here we are selecting right biased, then



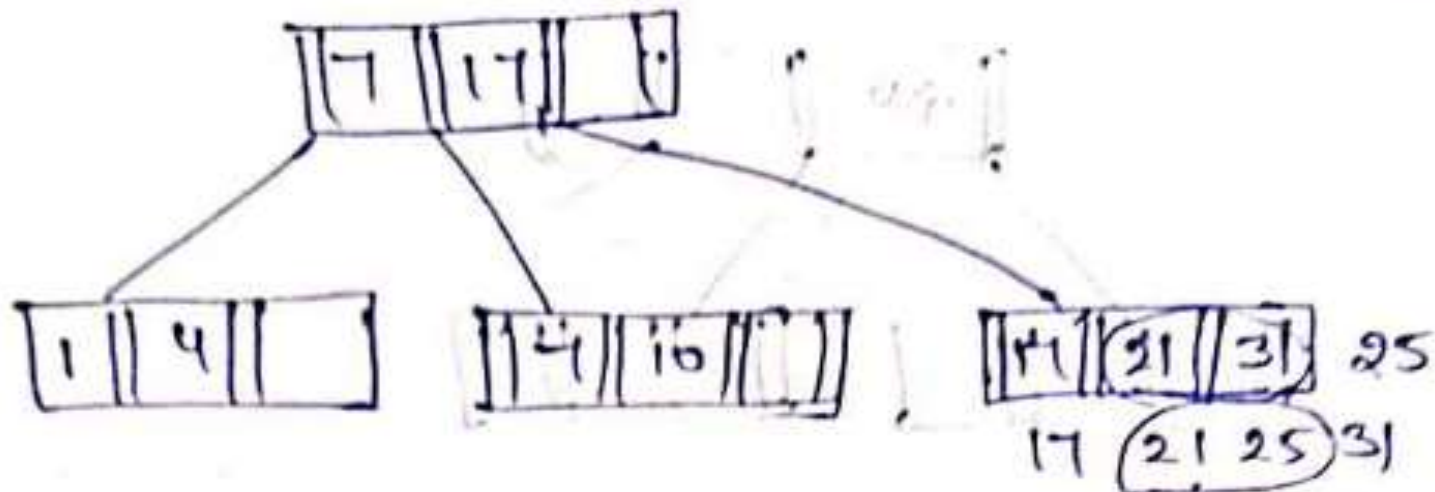
Insert: 17, 21



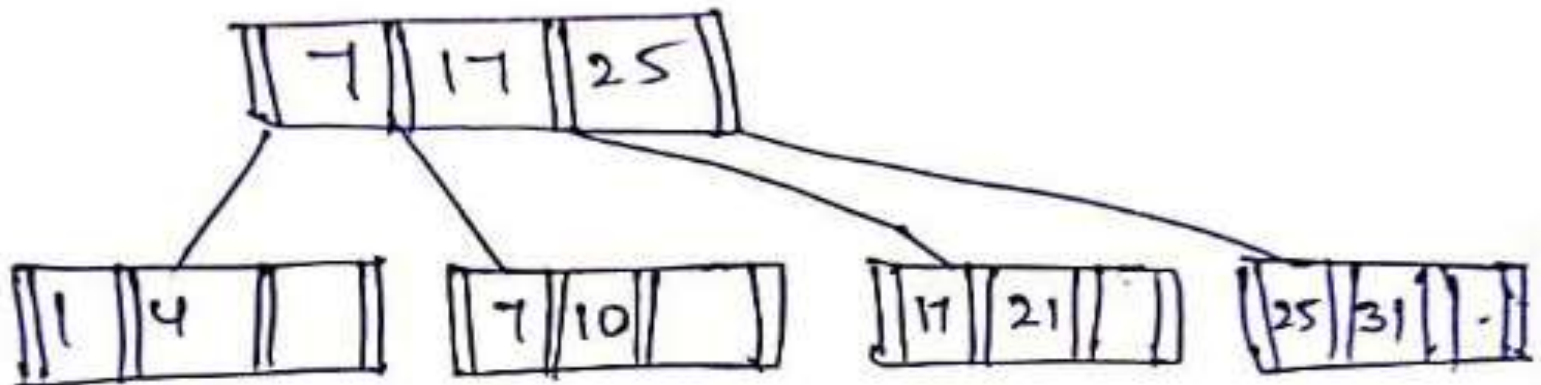
Insert: 21



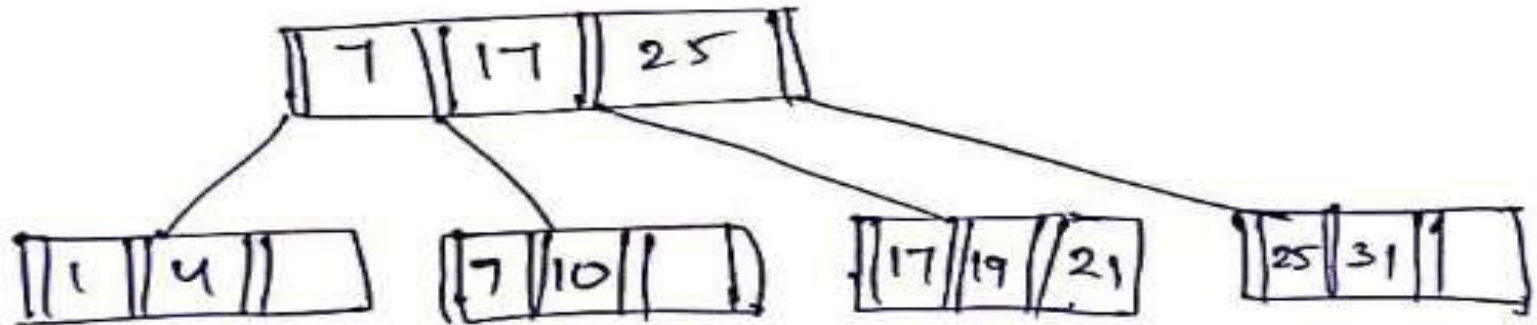
Insert : 31, 25 , 20 is already in the array



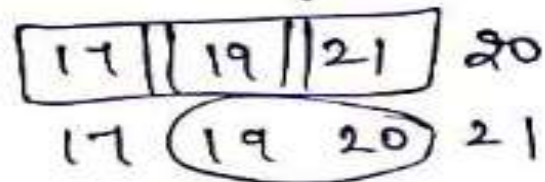
As we have taken right biased 25 moves to upper node as index value.



Insert 19, 20



for inserting 20, node is full



20, should be moved to upper node.

Now, upper node is

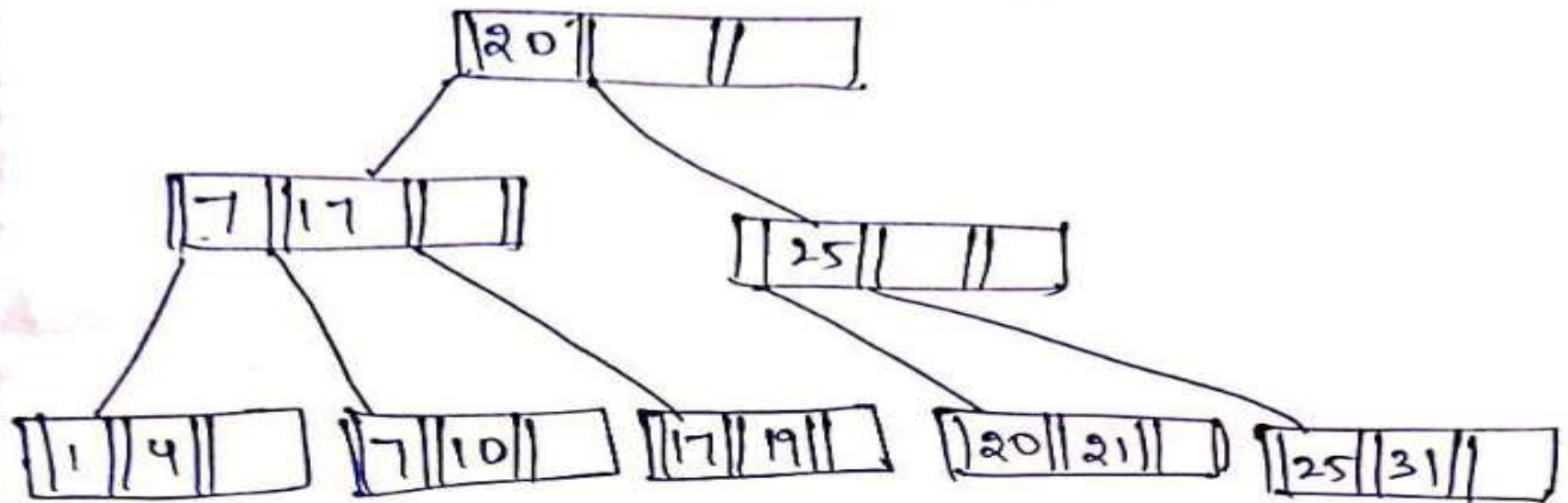
7	17	25	20
---	----	----	----

=> upper node is also full,

7	17	20	25
---	----	----	----

=> so, again 20 should be moved to one more level.

Now, the structure is as follows.



Insert: 28

Now, $28 > 20$ Right sub Tree (RST)

$28 > 25$ RST

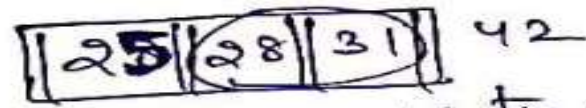
[25 | 31 | 28]

Node should be in sorting order.

[25 | 28 | 31]

Insert: 42

42 7 20 RST
42 7 25 RST
42 7 31 $\Rightarrow$



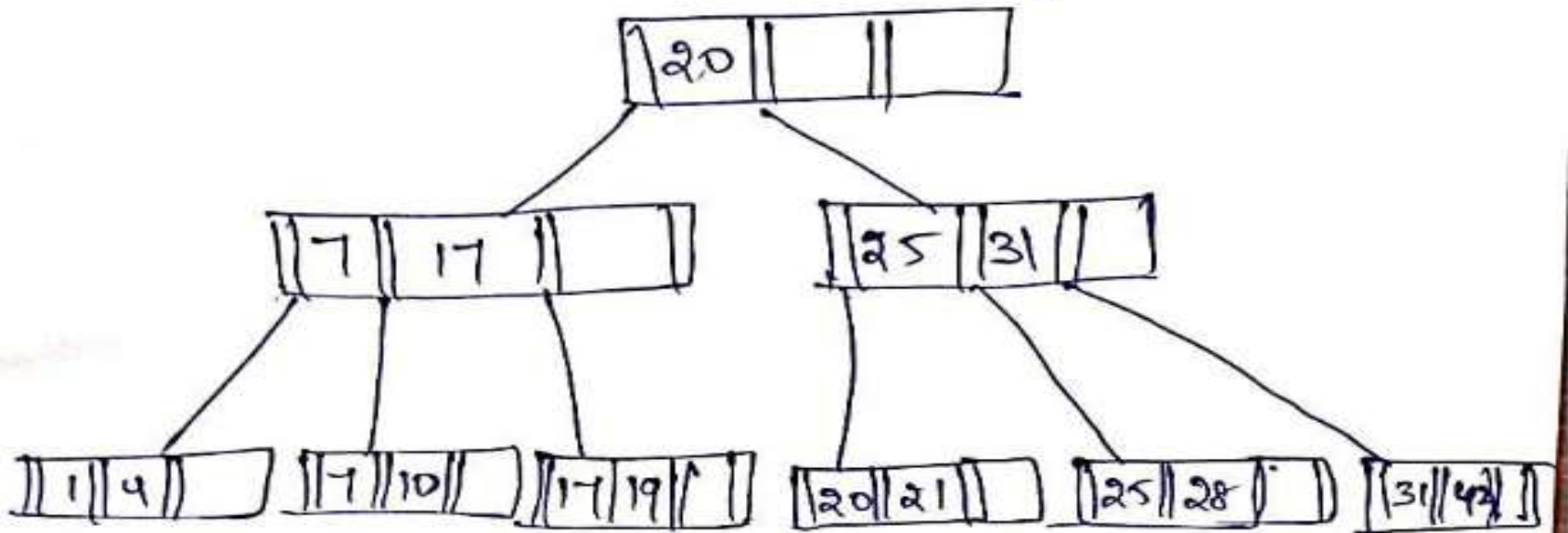
31 moves to upper node.

Now, leaf nodes are $\Rightarrow$

25	28	
----	----	--

31	42	
----	----	--

final structure, after inserting all elements.



Deletion of B+ Tree

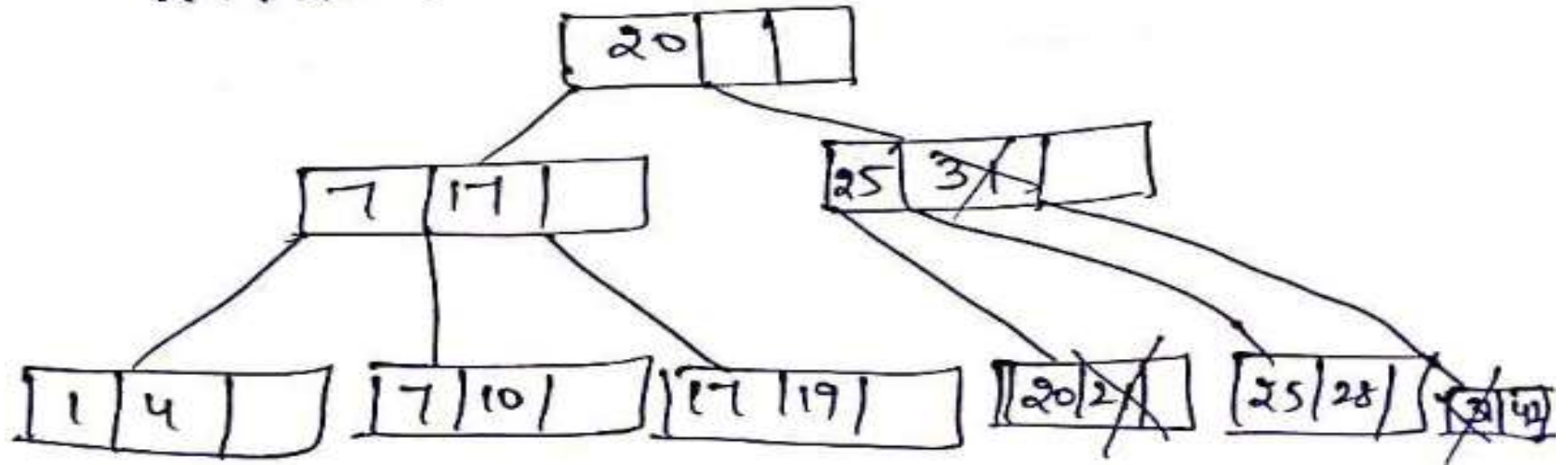
Delete: 21, 31, 20, 10, 7, 25, 42

Note:- When we are performing delete operation, we need to check whether node is containing minimum keys or not.

So, as considering above example we, assumed order: 4 i.e., min. key = 1

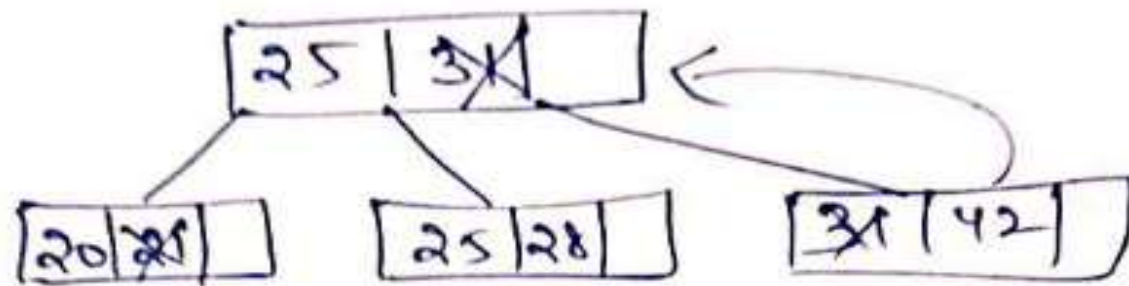
Delete: 21, 31

21 7 20 → Move to RST



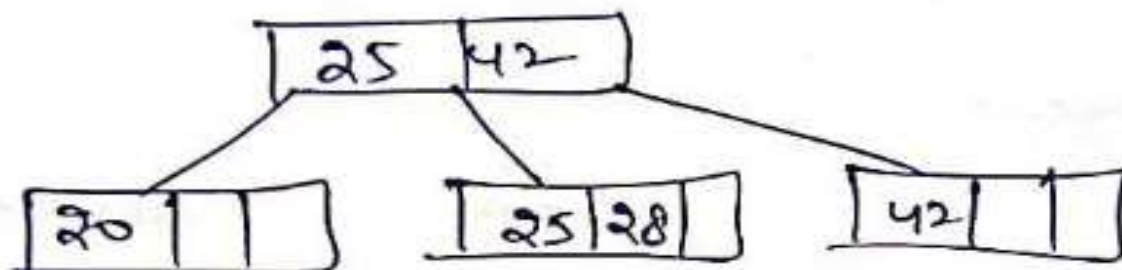
As 31, is removed in leaf node, we need to remove in index level node also.

⇒ so, index value node become as



42 moves to index node.

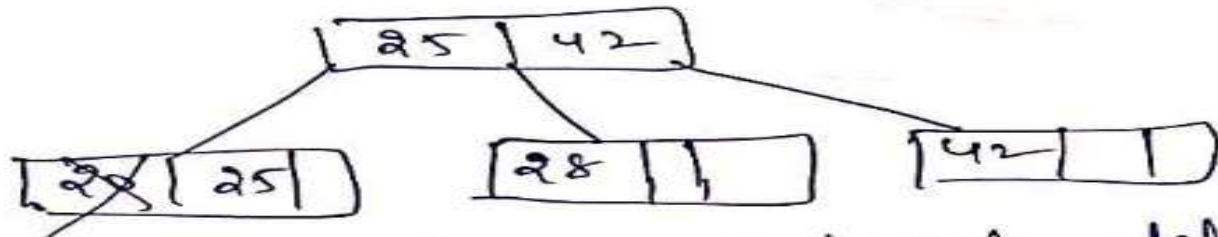
Now structure looks like,



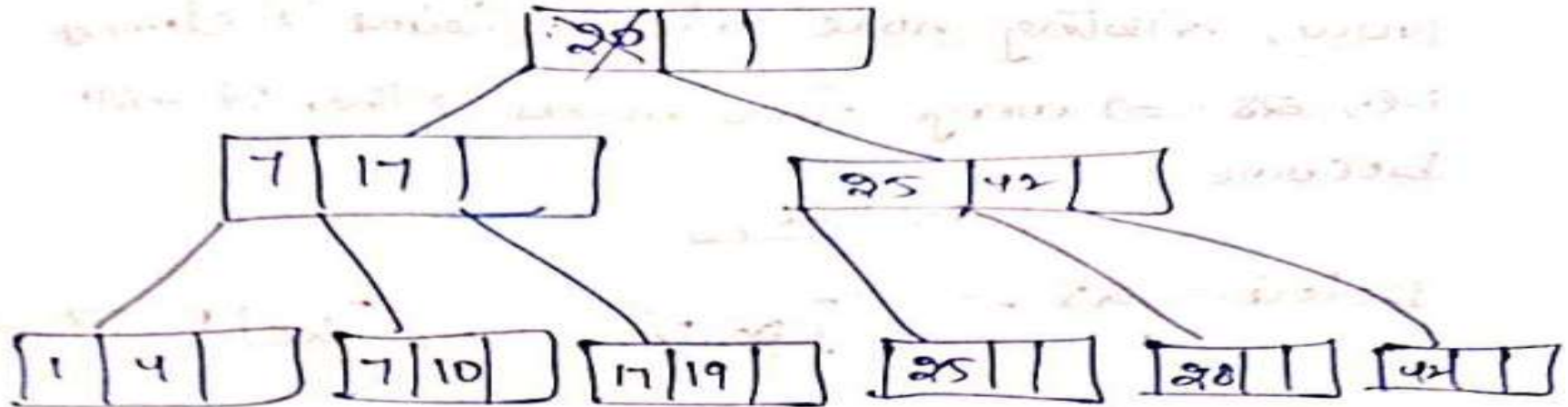
Delete ! 20

20, cannot be remove as that node contains one element. Now, we can borrow element from sibling node ~~either~~ either left side or right side.

let us borrow 25 from right side node.

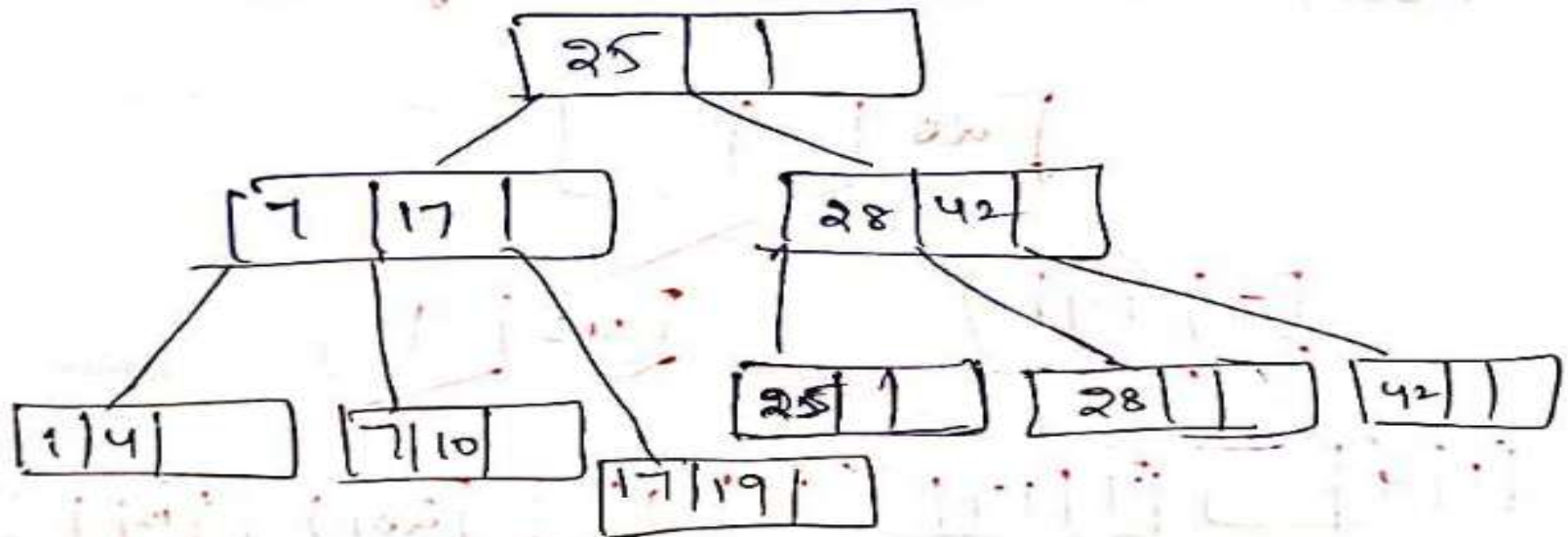


⇒ As 20, 25 also in root node, delete Root key element and replace other element as Root node.



→ when we removed 20 key in root node, replace root key value with smallest value in Right sub tree.

Here, 25 is smallest value.



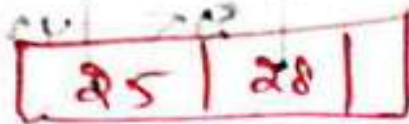
Delete : 10 ⇒



Delete : 25 ⇒

25 cannot be removed, as that contains minimum element '1'.

Now, sibling node also contains '1' element
i.e., 28 $\Rightarrow$ merge two nodes then it will
become

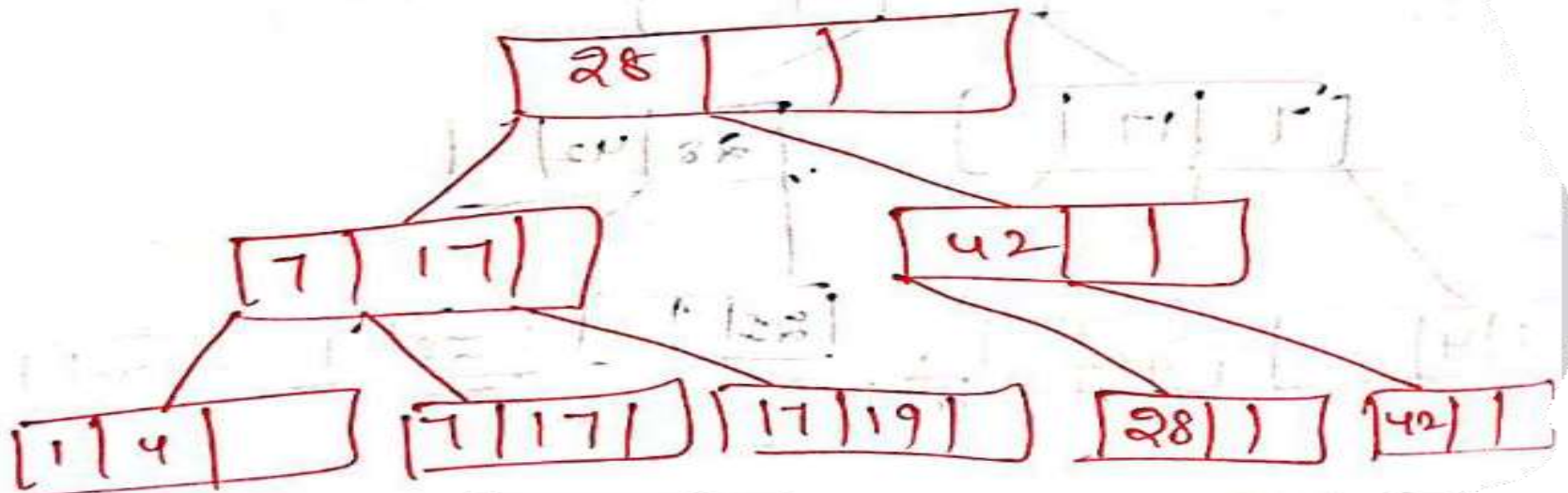


Delete - 25 :-



$\Rightarrow$ 25, also remove from root node
 $\rightarrow$ so, next smallest value is 28, above
moves to Root node

Now, the structure is as follows

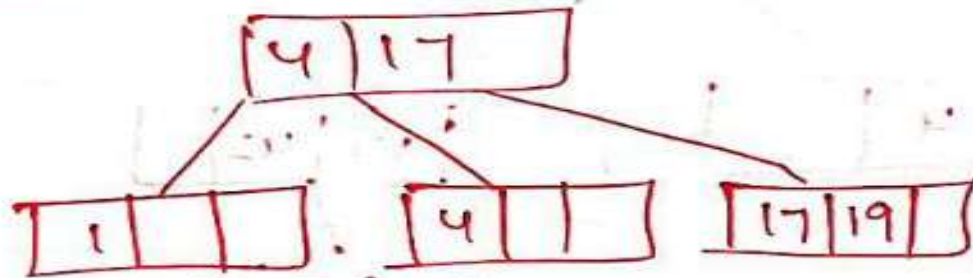
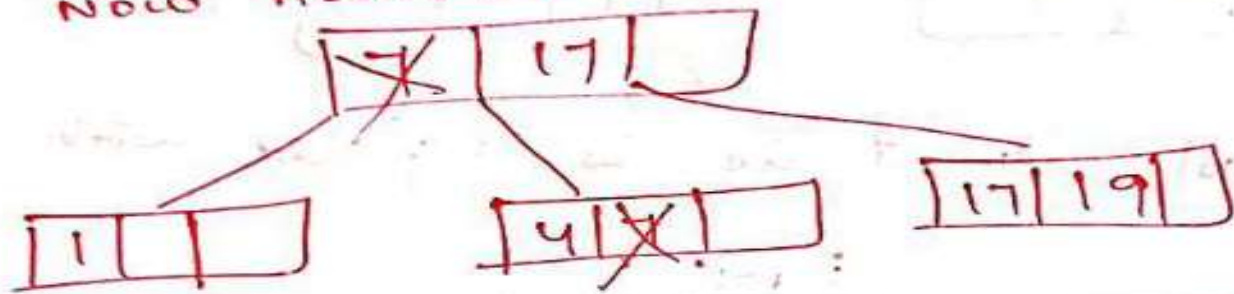


→ Delete: 7

7 also, single key in node, so, borrow from sibling node i.e, 4



Now nodes are



⇒ Delete: 42
 42 is single key in node, merge nodes
 i.e., 28 and 42 ⇒

28	1	
----	---	--

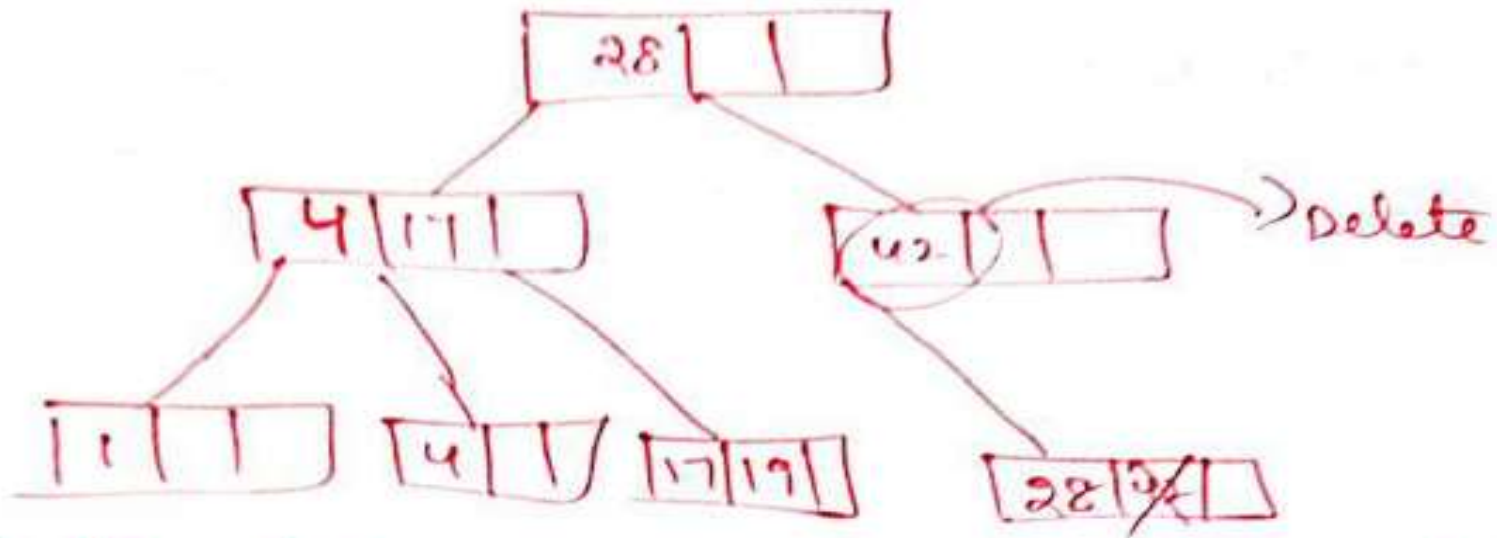
42	1	
----	---	--

 ⇒

28	42	
----	---------------	--

→ As 42 removed, even in index value
 42 also remove and replace with smallest
 element in Right subtree i.e., 28.

Now the structure is

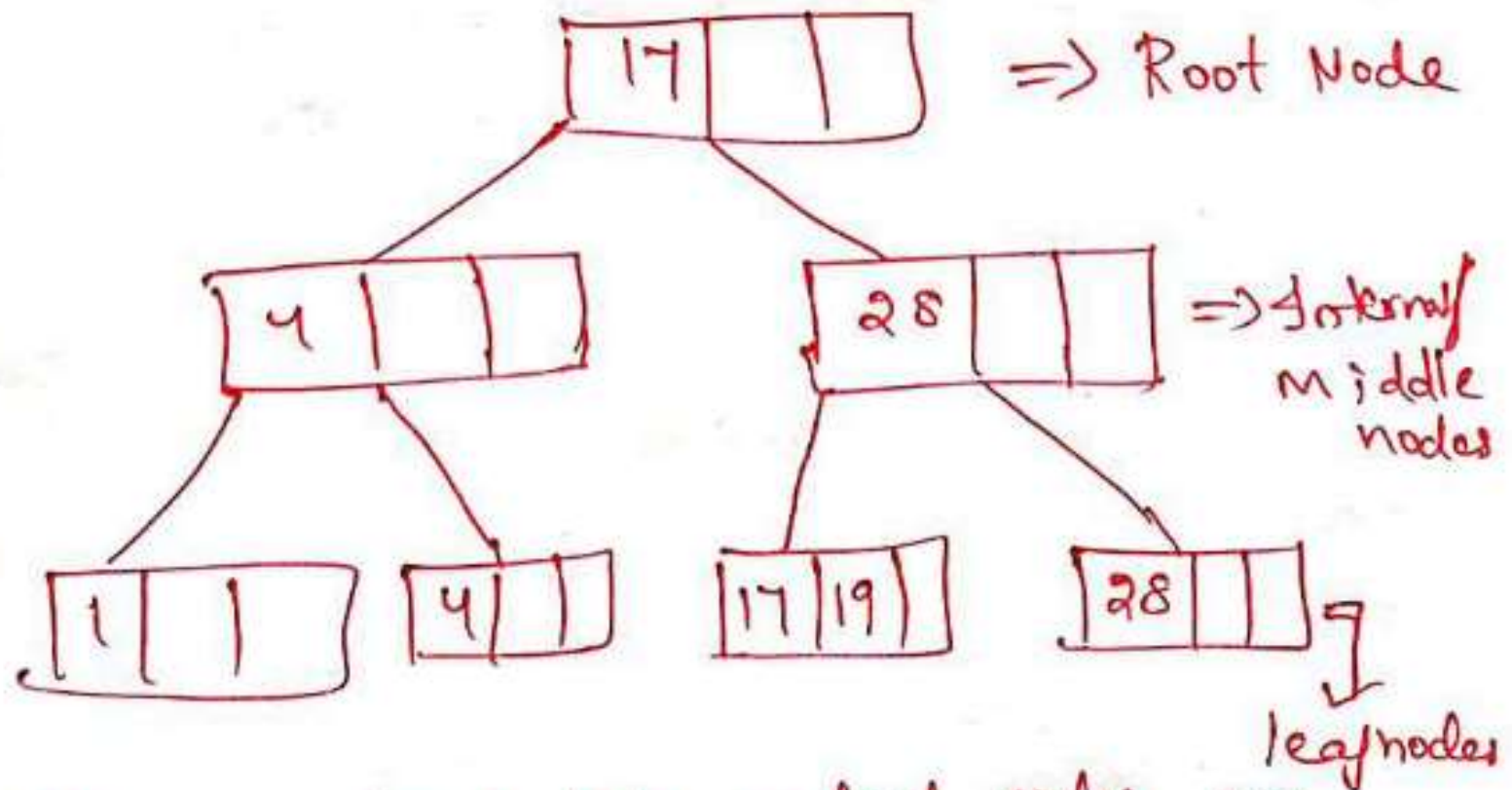


=> As when we removed 42 in Index node, it's violating condition as node should contain single element.

=> So, borrow element from sibling.

node [4 | 17 |] through root element becomes as it is left sub tree, then 17 ~~is~~ ^{becomes} a root element.

Now, final structure is as follows



Now actual data in leaf nodes are

1, 4, 17, 19, 28.

Advantages of B+Trees

- A B+ tree with 'l' levels can store more entries in its internal nodes compared to a B-tree having the same 'l' levels. This accentuates the significant improvement made to the search time for any given key. Having lesser levels and the presence of P_{next} pointers imply that the B+ trees is very quick and efficient in accessing records from disks.
- Data stored in a B+ tree can be accessed both sequentially and directly.
- It takes an equal number of disk accesses to fetch records.
- B+trees have redundant search keys, and storing search keys repeatedly is not possible.

Disadvantages of B+ Trees

- The major drawback of B-tree is the difficulty of traversing the keys sequentially. The B+ tree retains the rapid random access property of the B-tree while also allowing rapid sequential access.